Vrije Universiteit Amsterdam

Bachelor Thesis

The Impact of Compiler Optimizations on the Alignment Between LLM-Generated and Traditional Control-Flow Graphs

---

Author: Yavuz Kaya (2801089)

1st supervisor: Herbert Bos
2nd reader: Asia Slowinska

A thesis submitted in fulfillment of the requirements for the VU Bachelor of Science degree in Computer Science

July 22, 2026

The Impact of Compiler Optimizations on the Alignment Between LLM-Generated and Traditional Control-Flow Graphs

Yavuz Kaya
Vrije Universiteit Amsterdam
Amsterdam, Netherlands

Abstract

Control-Flow Graphs (CFGs) are a fundamental representation in reverse engineering and binary analysis, supporting applications such as vulnerability analysis, malware detection, and program comprehension. Recent advances in Large Language Models (LLMs) have demonstrated strong reasoning capabilities for software engineering tasks, yet their ability to reconstruct CFGs from binary disassembly remains largely unexplored.

We evaluate the capability of CFG reconstruction of three LLMs (Gemini, DeepSeek, and Grok) from binary disassembly and compare their performance with established analysis frameworks, namely LLVM, Ghidra, and Rizin. A benchmark dataset consisting of ten open-source C programs compiled with four optimization levels was used. The reconstructed CFGs were evaluated using graph metrics, including node count, edge count, cyclomatic complexity, strongly connected components, and graph density, complemented by a qualitative analysis of the models' reasoning.

The results show that traditional frameworks consistently produce more structurally complete CFGs, while the evaluated LLMs generate smaller and more abstract representations. Although the models demonstrate an understanding of fundamental CFG concepts, they currently lack the precision required for reliable binary CFG reconstruction.

Keywords

Large Language Models, Control-Flow Graphs, Compiler Optimizations, Program Analysis, Binary Analysis

1 Introduction

1.1 Motivation

As a fundamental program representation, Control Flow Graphs (CFGs) support the examination of execution paths and facilitate reasoning about program behavior. As previous studies have explored LLM-based CFG reconstruction [8], there is a lack of research investigating how compiler optimization levels influence the structural correspondence between LLM-generated CFGs and those produced by established analysis tools. Current LLM-based approaches are frequently constrained by reproducibility issues and tendencies toward hallucination[7]. Therefore, evaluating the impact of compiler optimizations on structural similarity is essential to determining the reliability of LLMs in program analysis.

1.2 Research Questions

To bridge these gaps, we investigate how compiler optimizations influence the relationship between LLM-generated and traditional control-flow graphs. We structure our evaluation around one primary research question (RQ) and three supporting sub-questions (SQs):

- **RQ:** How do compiler optimizations impact the structural similarity between LLM-generated CFGs and traditional CFG representations?
- **SQ1:** What baseline differences in graph size, complexity, and connectivity characteristics exist between control-flow graphs generated from compiler intermediate representation and binary reconstruction ?
- **SQ2:** How closely do LLM-generated graphs match traditional representations when code is unoptimized?
- **SQ3:** How do incremental compiler optimization levels alter the structural similarity between LLM-generated CFGs and traditional CFG representations relative to the unoptimized baseline?

1.3 Contributions

We present an evaluation of LLMs for CFG reconstruction from binary disassembly. It introduces a reproducible evaluation based on quantitative graph metrics to compare LLM-generated CFGs with those produced by established binary analysis frameworks. Furthermore, it provides a comparative analysis of three LLMs (Gemini, DeepSeek, and Grok) alongside LLVM, Ghidra, and Rizin. In addition to the quantitative evaluation, we investigate the reasoning employed by LLMs during CFG reconstruction, identifying common strengths and limitations in their handling of control flow.

2 Background

2.1 Control-Flow Graphs (CFGs)

Control-flow graphs (CFGs) are directed graphs where nodes represent basic blocks and edges represent potential control-flow transitions between them. A CFG is formally denoted as $G = (B, E)$, where B is the set of basic blocks and E is the set of directed edges.[1].

A basic block is defined as a linear sequence of program instructions containing a single entry point and a single exit point.[1].

Edges are expressed as ordered pairs (b_i, b_j), signifying a transfer of control from node b_i to node b_j. The source and destination nodes may be identical ($b_i = b_j$), representing self-loops within the graph [1].

Within a CFG, each node maintains a set of immediate successors and predecessors. Successor nodes represent blocks reachable directly via an outgoing edge, whereas predecessor nodes represent blocks with a direct incoming edge to the current node. Nodes may be categorized by their connectivity; for instance, an entry block typically possesses no predecessors, while a terminating block may possess no successors [1].

Understanding control flow is fundamental to program analysis, as numerous static analysis techniques rely on precise knowledge of possible execution paths. Hence, CFGs serve as a foundational representation for analyzing and reasoning about program behavior [1].

2.2 LLVM IR

LLVM is a compiler framework designed to simplify program analysis and transformation across compile-time, link-time, and run-time execution stages. To reinforce these processes, LLVM utilizes a common, low-level intermediate representation (IR) in Static Single Assignment (SSA) form. The IR incorporates a language-independent type system, which preserves sufficient structural information to optimize and analyze.[12]

Thus, LLVM IR is utilized in this study as the primary compiler-level representation from which control-flow graphs are extracted.

2.3 Binary Analysis

Binary analysis concerns the examination of compiled binary code without requiring access to the original source code. It is essential for vulnerability assessment, particularly when source code is unavailable and is broadly applied across various domains, including reverse engineering, debugging, performance optimization, security analysis, and digital forensics [9].

Ghidra and rizin serve as the binary analysis frameworks for extracting control-flow graphs from compiled executables. Unlike LLVM IR, Ghidra and rizin reconstruct control-flow graphs directly from binary code. Since binary CFG reconstruction is not fully standardized, different frameworks may adopt different definitions of basic blocks and interprocedural edges. These design choices produce different CFGs, even for the same binary, and should therefore be considered when selecting frameworks.

2.3.1 *Ghidra.*

Ghidra is a software reverse engineering framework developed and maintained by the National Security Agency (NSA) Research Directorate. Its core capabilities encompass disassembly, assembly, decompilation, graph visualization, and scripting. Moreover, Ghidra supports a diverse range of processor instruction sets and executable formats, offering flexibility for both interactive and automated reverse engineering tasks[14].

2.3.2 *rizin.*

Rizin is a free and open-source reverse engineering framework that provides a comprehensive environment for binary analysis. It prioritizes usability, stability, and functionality for both developers and users. The framework offers an advanced command-line interface that supports resource navigation, data analysis and visualization, disassembly, binary patching, data comparison, and search-and-replace operations [18].

2.4 LLMs

Large Language Models (LLMs) have rapidly become a dominant approach in natural language processing, demonstrating strong performance across diverse language understanding and generation tasks. The models derive their capabilities from large-scale pre-training on vast text corpora, often utilizing billions of parameters in accordance with established deep learning scaling laws.[13]

As model scale increases, these systems demonstrate capabilities that extend beyond language understanding and text generation, including emergent behaviors not commonly observed in smaller language models. Notable examples are in-context learning, instruction following, and multi-step reasoning, where complex tasks are addressed through intermediate reasoning steps[13].

Consequently, we investigate the capabilities of large language models in program analysis, with a specific focus on control-flow graph (CFG) reconstruction and its relationship to traditional program analysis techniques. To provide a representative evaluation across contemporary large language models, Gemini Flash Lite 3.1, DeepSeek-V3, and Grok 4 were selected.

2.4.1 *Gemini.*

Gemini 3.5 Flash is a natively multimodal reasoning model in Google's Gemini 3 series. It is designed to provide strong performance across a wide range of tasks while maintaining high computational efficiency. The model is particularly suited for agentic workflows, code generation and analysis, and long-running enterprise processes. [5]

2.4.2 *DeepSeek.*

DeepSeek-V3 employs an auxiliary-loss-free load balancing strategy and a multi-token prediction objective to further enhance model performance. According to the authors' evaluation, DeepSeek-V3 surpasses existing open-source language models and achieves performance comparable to state-of-the-art proprietary models.[3]

2.4.3 *Grok.*

Grok 4 was trained to effectively utilize external tools through reinforcement learning, enabling it to enhance its reasoning by leveraging resources such as a code interpreter and web browsing when solving tasks that are typically challenging for large language models.[25]

3 Overview

Figure 1 presents an overview of the design. The methodology consists of four main stages. First, a collection of open-source C benchmark programs is compiled using Clang at multiple

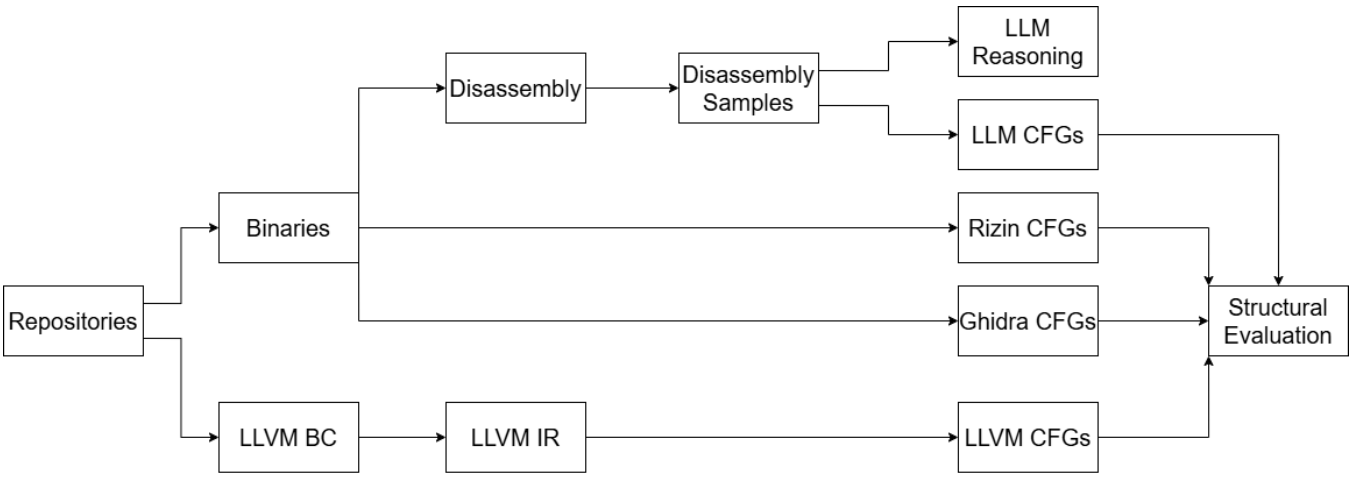

Figure 1: Overview of the design.

optimization levels to produce executable binaries and LLVM bitcode. The generated bitcode is linked and converted into human-readable LLVM intermediate representation (LLVM IR). Second, CFGs are recovered using three traditional program analysis approaches. LLVM IR is analyzed using LLVM to obtain compiler-level CFGs, Ghidra performs headless analysis of the compiled binaries to recover binary-level CFGs, and Rizin analyzes the same binaries using its `aaa` analysis command, with functions and their basic blocks extracted to reconstruct the corresponding CFGs. After that, each compiled binary is disassembled using `objdump`, then the resulting assembly listings are anonymized into disassembly samples. These samples are delivered individually to each evaluated large language model together with the CFG reconstruction prompt presented in Appendix A. Each model reconstructs a CFG and returns it in a standardized JSON format together with a confidence score and a textual explanation describing its reconstruction process and assumptions. Finally, all reconstructed CFGs are converted into a common graph representation and evaluated using graph-based structural metrics. Along with the structural evaluation, the confidence scores and reasoning generated by the large language models are collected and analyzed qualitatively to provide further insight into their reconstruction strategies and decision-making process.

4 Design

4.1 Control-Flow Graph Representation

Although control-flow graphs are widely used in binary analysis, their construction is not fully standardized across analysis frameworks. Different frameworks may adopt different definitions of basic blocks and control-flow edges, resulting in structurally different CFGs even when analyzing the same binary. These differences affect graph metrics such as the number of nodes, edges, cyclomatic complexity, and strongly connected components, making consistent CFG definitions essential for meaningful comparisons.

During the design phase, several binary analysis frameworks were evaluated. While Angr[20] is widely used in binary analysis research, its `CFGFast` analysis explicitly models function calls by introducing call and fake-return edges and frequently terminates basic blocks at call instructions. In contrast, LLVM, Ghidra, and Rizin follow the conventional basic block definition in which ordinary function calls remain within the current basic block when execution is expected to return.

Several normalization techniques were investigated to merge Angr's artificially split basic blocks. Although these reduced some structural differences, they could not fully reconcile Angr's CFG representation with the conventional representation adopted by the other frameworks. Consequently, Angr was replaced by Rizin, whose CFG representation is compatible with LLVM and Ghidra, allowing structural differences to be attributed to the reconstruction approaches rather than differences in CFG definitions.

A similar consideration motivated the exclusion of SVF[21]. Unlike the other evaluated frameworks, SVF operates on LLVM IR and constructs interprocedural control-flow graphs (ICFGs), whereas the objective of this study is to compare intraprocedural CFG reconstruction. Since LLVM provides intraprocedural CFGs and Ghidra reconstructs only intraprocedural CFGs from binaries, including SVF would have introduced a fundamentally different graph representation into the comparison. Therefore, we focus exclusively on intraprocedural CFG reconstruction to ensure methodological consistency.

4.2 Dataset Selection

4.2.1 *Selection Criteria.*

The programs were selected based on a set of criteria. Each program was required to function as a standalone application. To simplify the replication of the results, only open-source

software was considered. Programs were required to be compatible with LLVM and compilable using Clang. To mitigate domain-specific bias, the dataset comprises programs from a wide range of application domains. Preference was given to small-to-medium-sized programs, an approach that ensures the analysis remains computationally manageable while still capturing meaningful and complex control-flow structures. Additionally, the size of the generated disassembly was limited to approximately 900 KB to ensure that it could be processed reliably by the evaluated Large Language Models within their practical input constraints. Finally, the dataset was curated to include programs with diverse characteristics to effectively evaluate the impact of compiler optimizations across a broad range of complexities.

4.2.2 *Dataset Source.*

The dataset consists of ten open-source C programs obtained from publicly available GitHub repositories. For *bzip2*, *pdpmake*, and *wak*, publicly available single-file adaptations of the original projects were obtained from the GitHub repository fuhsnn/c_files. Table 1 lists the selected programs and references their original upstream repositories.

Table 1: Programs included in the dataset.

Program	Domain
lsh	Unix Shell
md5-c	Cryptographic Utility
2048.c	Game
fe	Expression Interpreter
kilo	Text Editor
Tinyhttpd	HTTP Server
Notepad–	Text Editor
bzip2	Compression Utility
pdpmake	Build Automation Tool
wak	AWK Interpreter

4.2.3 *Program Characteristics.*

Table 2: Characteristics of the selected programs.

Program	LOC	Functions	Loops	Conditionals
2048.c	508	19	26	41
Notepad–	874	54	19	145
Tinyhttpd	410	14	13	37
bzip2	5094	106	190	485
fe	662	48	11	31
kilo	986	36	26	159
lsh	176	10	6	21
md5-c	166	9	8	4
pdpmake	3187	86	101	431
wak	3787	195	87	497

To characterize the selected programs, metrics including the number of executable lines of C code (LOC), functions, loops, and conditional statements were collected for each entry. The LOC and number of functions collection process was automated using a Bash script designed to process each program repository. The script recursively identified all relevant C source files (`.c`) associated with each program.

Executable lines of code were measured using the `cloc` tool, which counts executable code lines while excluding comments and blank lines.

The number of functions was obtained using `ctags`, which was used to identify C function definitions, and the resulting counts were aggregated across all source files belonging to a program.

Loop counts included occurrences of `for` and `while` constructs, with `do-while` loops counted through their terminating `while` statement to avoid double counting. Conditional counts included `if`, `else`, `goto`, and `switch` statements, with `else-if` loops counted through their `if` statement to avoid double counting. Before counting, the script removed comments and string and character literals from the source code to reduce false positives caused by keyword occurrences in non-code regions. This approach provides more accurate measurements than simple keyword matching while remaining fully automated.

The collected statistics were used to ensure that the dataset comprises programs with varying sizes and control-flow characteristics, thereby providing a diverse benchmark for evaluation. The programs are small to medium-sized, ranging from 166 to 986 lines of code. This scale was chosen intentionally, as smaller programs reduce inconsistencies that may arise from complex multi-file projects or external library dependencies. While this controlled setting strengthens the internal validity of the study, it may limit the generalizability of the findings to substantially larger software systems, which remains a valuable avenue for future work.

4.3 GitHub Repository

To support the reproducibility of this study, the complete experimental pipeline, source code, datasets, the prompt, generated control-flow graphs, and evaluation results are publicly available in the following GitHub repository:
https://github.com/yavuzk28/Bachelor-Thesis

4.4 Compilation

To ensure consistency, a standardized pipeline was written in Python and was used to generate binaries and LLVM intermediate representations.

All relevant C source files associated with each program were identified recursively. Files related to testing, benchmarking, demonstrations, examples, and cleaning utilities were excluded to only include the primary program implementations. Subsequently, the identified source files were compiled using Clang under four distinct optimization levels: `-O0`, `-O1`, `-O2`, and `-O3`. These increasing optimization levels were selected to systematically investigate how compiler-driven transformations influence the structure of the resulting control-flow graphs.

For each optimization level, executable binaries were generated. Following compilation, the `strip` utility was used to

remove symbol information to replicate real-world deployment scenarios.

In parallel with binary generation, LLVM bitcode files (`.bc`) were produced using Clang's LLVM backend with the `-emit-llvm` option. For programs comprising multiple source files, the generated bitcode files were merged into a single LLVM module using `llvm-link`. This linked bitcode module was translated into human-readable LLVM intermediate representation (`.ll`) using `llvm-dis`.

The binaries and LLVM IR files were stored for each program and optimization level, serving as the primary input for the control-flow graph extraction stage.

All generated binaries were disassembled using the GNU `objdump` utility. The disassembly process was configured to use Intel syntax (`-Mintel`) and to omit raw instruction bytes (`--no-show-raw-insn`) to enhance readability. For each binary, the resulting assembly code was exported to a dedicated text file.

The disassembly process was automated using Python scripts. It recursively identified all generated binaries while excluding LLVM bitcode (`.bc`) and LLVM IR (`.ll`) files. Each binary was processed via `objdump`, and the assembly listings were cleaned by removing non-essential file-format headers. They served as the primary input representation provided to the large language models for control-flow graph reconstruction.

4.5 CFG Extraction

LLVM IR was utilized to obtain compiler-level CFGs. Additionally, Ghidra and rizin were used to reconstruct CFGs directly from compiled binaries. The graphs were converted into a standardized JSON representation consisting of nodes and directed edges.

Prior to LLM evaluation, anonymized assembly files were provided to the large language models for CFG generation. This anonymization was performed to prevent the models from inferring a program's identity or its compiler optimization level from filenames or associated metadata.

4.5.1 *LLVM.*

LLVM IR[12] is the intermediate representation generated by the LLVM compiler. It captures the compiler's perspective of program control flow through basic blocks connected by explicit control-flow instructions, providing a compiler-level representation of the program prior to binary generation.

During compilation, each source file was first translated into LLVM bitcode (`.bc`) using Clang. For programs consisting of multiple source files, the individual bitcode modules were linked into a single LLVM module using `llvm-link`. The linked module was subsequently converted into human-readable LLVM IR (`.ll`) using `llvm-dis`, producing a single LLVM IR file representing the complete program for each optimization level.

The extraction process identified LLVM basic blocks by locating block labels within each function definition, with each block represented as a node in the control-flow graph. Control-flow relationships between these blocks were recovered by analyzing LLVM branch instructions. Specifically, conditional branch instructions (`br`) produced two outgoing edges corresponding to the true and false execution paths, while unconditional branches resulted in a single outgoing edge to the target block. Notably, `goto` statements from the original C source are lowered by Clang into LLVM branch instructions during compilation, ensuring they are naturally captured within the extracted control-flow graphs.

In addition, `switch` instructions were analyzed to capture multi-way control-flow transfers. For each switch statement, edges were created from the originating basic block to both the default destination and all case destinations. Finally, sequential edges between adjacent basic blocks were added when the preceding block did not contain a terminating control-flow instruction, reflecting the implicit fall-through present in the LLVM IR.

Function termination instructions, including `ret`, `resume`, and `unreachable`, were treated as terminal nodes and therefore did not generate outgoing control-flow edges. The extraction process reconstructs only the explicit intra-procedural control flow encoded in the LLVM IR by analyzing branch and switch instructions. Since no pointer or indirect-call analysis is performed, control-flow edges arising from indirect function calls through function pointers are not included in the extracted CFG.

4.5.2 *Ghidra.*

Ghidra[14] is a reverse engineering framework that performs static analysis on binaries to recover program structure without requiring source code. It reconstructs control-flow graphs by disassembling machine instructions, identifying basic blocks, and resolving the control-flow relationships between them.

Binary control-flow graph recovery commonly relies on conservative over-approximation because indirect branch targets cannot always be resolved precisely through static analysis.[15] Accordingly, Ghidra reconstructs control-flow graphs using static binary analysis and may conservatively recover additional feasible edges in instances where precise branch targets cannot be statically resolved

For each program and optimization level, the generated binary was imported into a Ghidra project and processed using Ghidra's headless analysis framework. The extraction was performed via a Python script, which invoked a Java-based Ghidra analysis script to generate the corresponding control-flow graphs.

CFG extraction was performed using Ghidra's `BasicBlockModel`, which identifies basic blocks and their control-flow relationships within the analyzed program. The process iterated through all discovered functions, extracting the basic blocks associated with each. Each basic block was represented as a node in a graph. Directed edges were created for all recovered intra-procedural control-flow transfers, while interprocedural call edges were excluded to preserve function-local control-flow graphs.

For every identified basic block, the destinations of all outgoing non-call control-flow references were collected. These references were used to reconstruct the control-flow structure recovered by Ghidra's binary analysis framework.

The extracted nodes and edges were subsequently exported to a standardized JSON representation.

4.5.3 *Rizin.*

Rizin [18] is a free and open-source reverse engineering framework that provides a comprehensive environment for binary analysis. The framework offers a command-line interface with functionality for tasks such as binary analysis, disassembly, debugging, and binary inspection.

For each program and optimization level, the generated executable binary was analyzed using Rizin. The extraction was performed via a Python script using the `rzpipe` interface to interact programmatically with Rizin. Each binary was opened in Rizin and analyzed using the `aaa` command to perform automatic program analysis and recover functions and their associated control-flow information.

Following the analysis, the list of recovered functions was obtained using the `aflj` command. For each recovered function, its basic blocks were retrieved using `afbj`. Each basic block was represented as a CFG node and identified by its recovered function name and basic-block address. Intraprocedural control-flow edges were constructed from the `jump` and `fail` targets reported for each basic block. Only targets corresponding to basic blocks within the same recovered function were retained, ensuring that the resulting graphs represent intraprocedural control flow.

The extracted nodes and edges were subsequently exported to a standardized JSON representation containing the total node and edge counts, as well as the complete lists of recovered CFG nodes and intraprocedural control-flow edges.

4.6 LLMs

Large language models[13] were evaluated on their ability to reconstruct control-flow graphs from disassembly files, which were selected as the input for evaluation because directly analyzing binary data does not fully leverage the capabilities of LLMs, requiring numerous potential instruction addresses to be examined and consequently increasing the number of LLM queries, computational cost, and latency[19].

Prior to evaluation, the disassembly files were anonymized. Each assembly file and its corresponding binary were assigned a unique sample identifier, while program names and optimization-level information were stored separately in a mapping file. This anonymization process prevented the evaluated LLMs from inferring the identity of the benchmark program or the applied compiler optimization level from filenames or associated metadata.

Each sample consisted of a single disassembled program representation and was submitted individually to each selected LLM together with the same reconstruction prompt. Consequently, every evaluated model processed each anonymized disassembly file individually under identical prompting conditions. The prompt instructed each model to reconstruct a control-flow graph and return the output as a structured JSON object. The returned JSON contained a confidence score, a brief explanation of the reconstruction process, and the reconstructed CFG consisting of a set of nodes and directed edges.

To standardize the generated graphs, the prompt required basic blocks to be represented using a common naming convention (`<function>::B0`, `<function>::B1`, ...), where each node corresponded to a basic block within a function and each edge represented a possible intraprocedural control-flow transition. The models were instructed to rely exclusively on the provided program representation and to ignore filenames, metadata, symbols, comments, or any external information. When a complete reconstruction was not possible, the models were instructed to generate the portion of the CFG that could be inferred with reasonable confidence.

Finally, the JSON responses were parsed to extract the reconstructed CFGs, which were converted into the same graph representation used by the LLVM, Ghidra, and rizin.

The complete prompt used for all evaluated LLMs is provided in Appendix A.

4.7 Metrics

All graph-based metrics were computed using directed graph representations constructed with the Python NetworkX library [6]. NetworkX was utilized to compute graph properties, including strongly connected components and graph density, while additional measures such as cyclomatic complexity[4] were derived from these graph representations.

4.7.1 *Graph Size Metrics.*

Graph size metrics were employed to compare the overall scale of the generated control-flow graphs. The total count of nodes and edges was collected for each graph, where nodes represent basic blocks and edges represent the possible control-flow transitions between them.

- Node Count: Number of basic blocks contained in the control-flow graph.

- Edge Count: Number of directed control-flow transitions between basic blocks.

4.7.2 *Complexity Metrics.*

Cyclomatic complexity was employed to estimate the structural complexity of each control-flow graph, serving as a measure of the number of linearly independent execution paths through the program[4].

- Cyclomatic Complexity: Measures the number of linearly independent execution paths through the control-flow graph.

4.7.3 *Connectivity Metrics.*

Connectivity metrics were utilized to characterize the structural organization of each control-flow graph by measuring the presence of cyclic regions and the overall level of graph interconnectedness.

- Strongly Connected Components: Number of maximal subgraphs in which every node is reachable from every other node.

- Graph Density: Ratio of the number of directed edges to the maximum possible number of directed edges in the graph.

4.8 Experimental Procedure

The experimental evaluation was designed to address the research subquestions defined in the introduction.

For the first research subquestion, the CFGs extracted by the traditional analysis tools were compared with one another. CFGs produced by LLVM, Ghidra, and rizin were evaluated using the selected graph-based metrics to quantify the structural similarities and differences between the generated graphs.

The second research subquestion evaluated the ability of the large language models to reconstruct program control flow. To achieve this, the CFGs generated by the evaluated LLMs were compared against the reference CFGs extracted by LLVM, Ghidra, and rizin from the unoptimized (`-O0`) program versions. Using unoptimized binaries minimizes the influence of compiler optimizations and allows the comparison to focus on the models' reconstruction capabilities.

The third research subquestion investigated the impact of compiler optimizations on CFG reconstruction. For each optimization level (`-O1`, `-O2`, and `-O3`), the CFGs generated by the large language models were compared with the corresponding CFGs extracted by LLVM, Ghidra, and rizin from binaries compiled using the same optimization level. The results were analyzed to determine how compiler optimizations affect the structural correspondence between LLM-generated CFGs and those produced by traditional program-analysis techniques.

For each graph metric, boxplots were constructed to summarize the distribution of values produced by each traditional framework and large language model. The plots visualize the median, interquartile range, whiskers, outliers, and mean for each tool. A logarithmic y-axis was employed to facilitate comparison between traditional frameworks and LLMs, whose metric values differ by order of magnitude.

Additionally, for each comparison, the selected graph-based metrics were computed and analyzed. The confidence scores and reasoning provided by the evaluated large language models were also collected and analyzed to provide qualitative insight into the models' reconstruction process.

Together, these results form the basis to answer the research subquestions.

5 Evaluation

The y-axis is logarithmic to demonstrate the substantial differences in values between traditional and LLM-generated CFGs Unless otherwise stated, comparisons are based on the median values shown in the boxplots.

5.1 Node Count

Figure 2 presents the distribution of node counts. Traditional analysis frameworks consistently produce substantially higher node counts than the LLMs at every optimization level. Among the traditional frameworks, Ghidra consistently reconstructs the largest CFGs, followed by Rizin, while LLVM produces the smallest node counts. The only notable exception is a single outlier observed for LLVM at the O0 optimization level. Among the LLMs, Grok consistently generates the smallest CFGs across all optimization levels. Gemini generally produces the largest node counts, with the exception of O1, where DeepSeek slightly exceeds Gemini. Gemini also exhibits noticeably greater variability than the other LLMs, as reflected by its larger number of outliers. In contrast, DeepSeek and Grok each exhibit only a single outlier throughout the evaluation.

5.2 Edge Count

Figure 3 presents the distribution of edge counts. Similar to the node count results, the traditional analysis frameworks consistently produce substantially higher edge counts than the LLMs. The same ordering is observed among the traditional frameworks, with Ghidra producing the largest edge counts, followed by Rizin, while LLVM consistently reconstructs the smallest CFGs in terms of edges. The hierarchy among the LLMs is also largely preserved. Grok consistently generates the smallest edge counts, whereas Gemini generally produces the largest. The only exception occurs at the O3 optimization level, where DeepSeek slightly exceeds Gemini. Unlike the node count results, however, all three LLMs exhibit multiple outliers. DeepSeek and Gemini each produce three outliers across the evaluated optimization levels, while Grok exhibits two.

5.3 Cyclomatic Complexity

Figure 4 presents the distribution of cyclomatic complexity. Similar to the graph size results, the traditional analysis frameworks consistently produce higher cyclomatic complexity than the LLMs. The same hierarchy observed for node and edge counts is preserved among the traditional analysis frameworks. Ghidra consistently produces the highest cyclomatic complexity, followed by Rizin, while LLVM yields the lowest values across all optimization levels. Among the LLMs, Gemini consistently generates the highest cyclomatic complexity, whereas Grok produces the lowest. DeepSeek falls between the two models. Regarding variability, both Gemini and DeepSeek exhibit a relatively large number of outliers, indicating greater variation in the complexity of their reconstructed CFGs. In contrast, Grok exhibits only a single outlier throughout the evaluation, suggesting more consistent cyclomatic complexity across the benchmark programs.

5.4 Strongly Connected Components

Figure 5 presents the distribution of strongly connected component counts. Similar to the graph size and complexity results, the traditional analysis frameworks consistently produce substantially higher SCC counts than the LLMs. The same ordering observed in the previous metrics is maintained

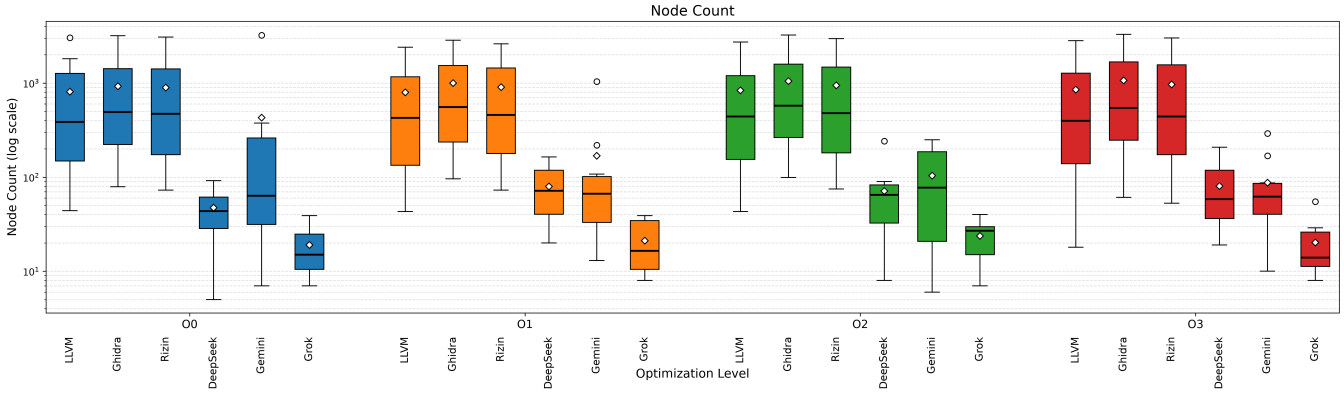

Figure 2: Distribution of node counts

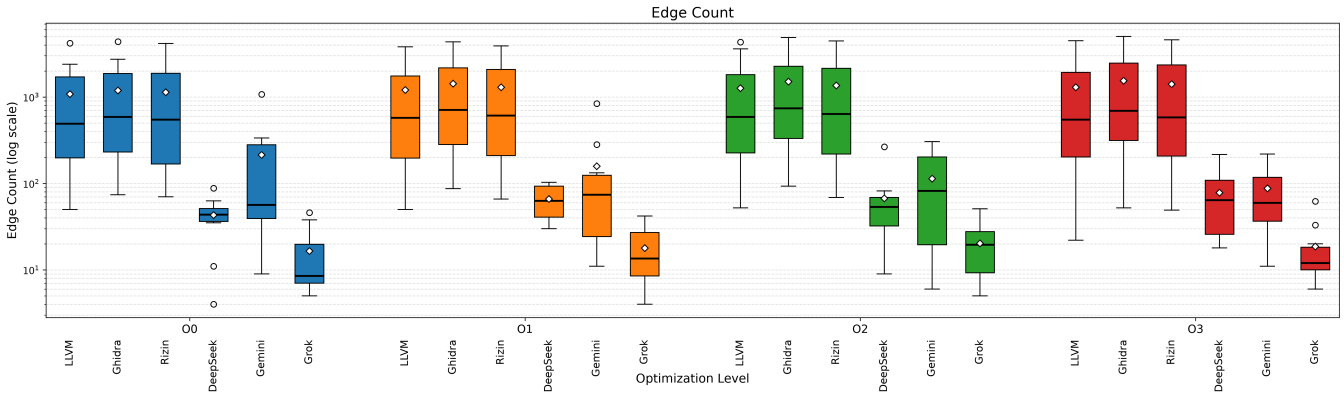

Figure 3: Distribution of edge counts

among the traditional analysis frameworks, with Ghidra consistently producing the highest SCC counts, followed by Rizin, while LLVM reconstructs the fewest strongly connected components. Unlike the previous metrics, however, all three traditional frameworks exhibit multiple outliers. The hierarchy among the LLMs is also preserved, with Gemini consistently producing the highest SCC counts, DeepSeek occupying the middle position, and Grok generating the lowest values. In terms of variability, Gemini exhibits noticeably more outliers than the other two LLMs.

5.5 Graph Density

Figure 6 presents the distribution of graph density. Unlike the previous four metrics, the ranking of both the traditional analysis frameworks and the LLMs differs considerably. Among the traditional frameworks, LLVM consistently produces the highest graph density, followed by Rizin, while Ghidra reconstructs the least dense CFGs, effectively reversing the ordering observed for node count, edge count, cyclomatic complexity, and strongly connected components. A similar reversal is observed among the LLMs. Grok consistently generates the highest graph density, followed by Gemini and DeepSeek. The only exception occurs at the O0 optimization level, where Gemini and DeepSeek exchange positions. In terms of variability, LLVM exhibits a noticeably larger number of outliers at the O3 optimization level compared to the other optimization levels.

6 Discussion

6.1 Traditional CFGs (SQ1)

Traditional CFG extraction approaches produce similar control-flow graphs throughout the evaluated metrics. The relative ordering of the tools remains largely consistent, although slight differences exist in graph size, connectivity, and complexity.

Determining an absolute ground truth for binary CFG recovery remains an open challenge. Therefore, observed differences should not be interpreted as errors, but as the result of differing analysis methodologies and design goals.[16]

For this study, we treat binary analysis tools (Rizin and Ghidra) as the practical reference for evaluating LLM performance, while LLVM serves as a compiler-level baseline.

The binary analysis tools generally reconstruct slightly larger CFGs than LLVM, with slightly higher node counts,

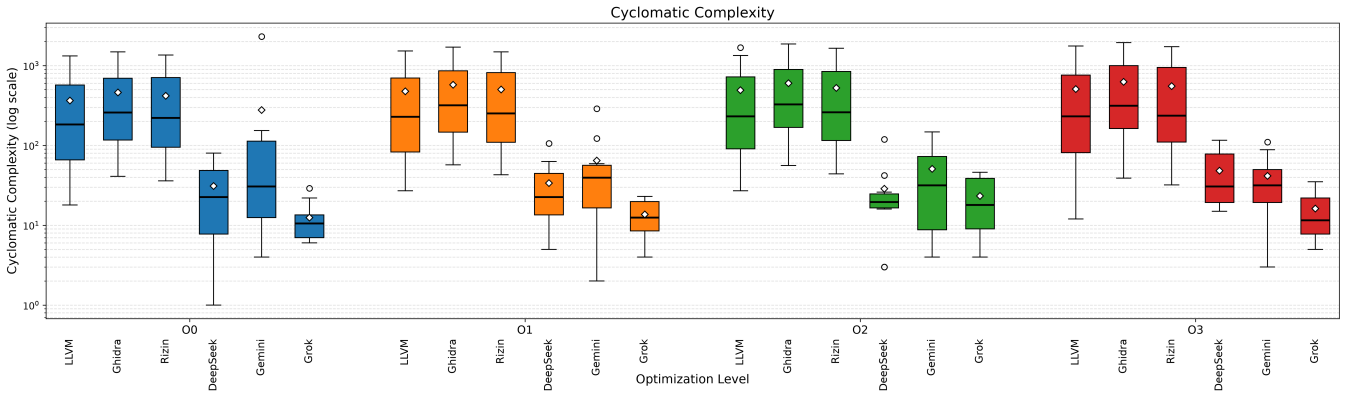

Figure 4: Distribution of cyclomatic complexity

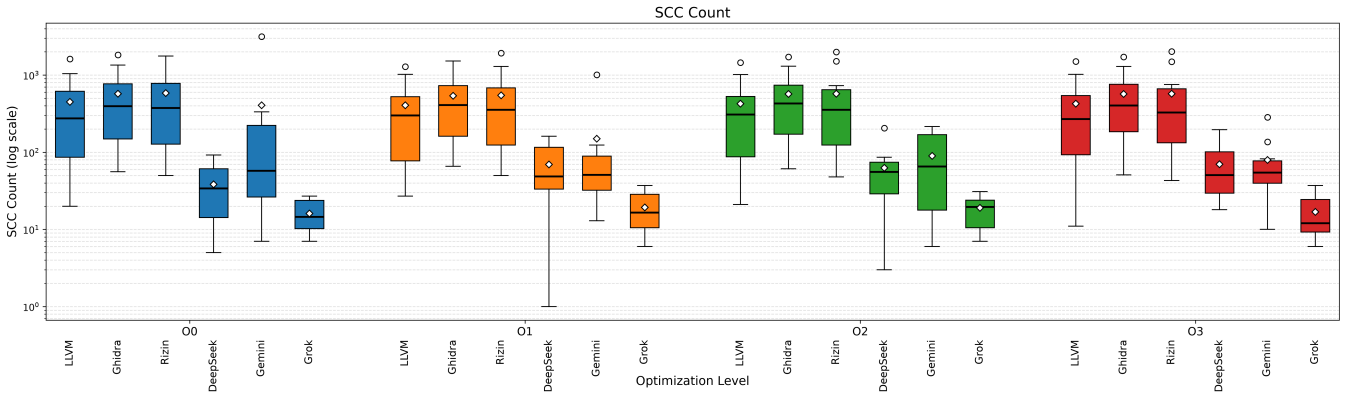

Figure 5: Distribution of strongly connected components

edge counts, cyclomatic complexity, and SCC counts. The observed ordering suggests that these differences are systematic characteristics of the respective analysis approaches.

This is expected, as both tools perform static binary analysis and thus conservatively over-approximate feasible control-flow paths when precise branch targets cannot always be resolved statically.[15] In contrast, LLVM generally produces slightly smaller and less complex CFGs than Ghidra and Rizin while exhibiting slightly higher graph density. Because LLVM operates on compiler IR before binary generation, it does not face the uncertainty in recovering control flow from binaries.

Based on these observations, the CFGs produced by the binary analysis frameworks can be regarded as conservative over-approximations of compiler-level CFGs. Consequently, the accuracy of LLM-generated CFGs is evaluated based on how closely they replicate the structural characteristics of these over-approximated CFGs, while LLVM serves as a compiler-level baseline for comparison.

6.2 LLM Results (SQ2)

A substantial gap remains between LLM-generated CFGs and the control-flow graphs recovered by traditional tools. For node count, edge count, cyclomatic complexity, and strongly connected components, all evaluated LLMs produce considerably smaller and less complex CFGs than the traditional frameworks. One possible explanation is that current LLMs prioritize semantically meaningful and human-readable code over preserving low-level binary details such as auxiliary branches and indirect transfers [22].

Consistent with this interpretation, Gemini and DeepSeek generally produce the most detailed CFGs among the LLMs, though still well below Rizin and Ghidra. This might indicate that they preserve more low-level control-flow information during reconstruction, whereas Grok favors more abstract program representations.

Interestingly, Grok generally exhibits higher graph density than the other LLMs. However, this is a byproduct of its smaller graphs rather than superior recovery.

While LLMs show solid high-level understanding, they currently lack the granularity required for accurate binary CFG reconstruction. These findings align with prior studies

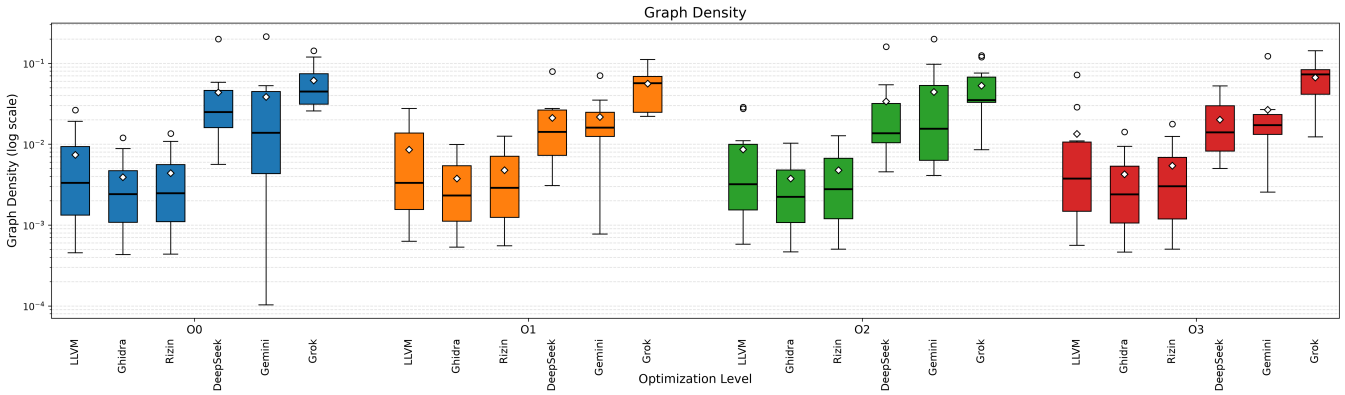

Figure 6: Distribution of graph density

indicating that LLMs are promising assistants but not yet reliable for fully automated zero-shot reverse engineering [10, 17]. Taken together, the results imply that current LLMs are better suited as assistants to reverse engineers during high-level program comprehension than for replacing traditional frameworks.

6.3 Compiler Optimizations (SQ3)

Compiler optimization has only a limited influence on the relative behavior of the evaluated approaches. Traditional frameworks consistently produce more detailed CFGs than the LLMs. Additionally, the LLMs exhibit greater variability, as indicated by the larger number of outliers, which is likely attributable to their non-deterministic generation process. Furthermore, the relative ordering remains stable across optimization levels, with Ghidra > Rizin > LLVM among the traditional frameworks and Gemini > DeepSeek > Grok among the LLMs.

The limited influence of compiler optimization suggests that the observed performance differences primarily reflect the inherent reconstruction capabilities of the evaluated approaches rather than their sensitivity to optimization-specific binary transformations. This should not be interpreted as robustness. Instead, it likely reflects the models' limited ability to capture detailed control-flow structures regardless of optimization level, with their reconstruction performance being dominated by limitations in low-level binary reasoning. This observation is consistent with previous work showing that current LLMs struggle with precise instruction-level reasoning despite demonstrating a sound understanding of higher-level program structures [10].

Although graph density exhibits a different ranking than the other evaluated metrics, the overall structural trends remain largely consistent. In particular, the evaluated LLMs exhibit a tendency toward generating more abstract control-flow graphs than traditional binary analysis frameworks.

These findings suggest that improvements in LLM-based CFG reconstruction are more likely the result of advances in low-level binary reasoning than increased robustness to compiler optimization.

6.4 Qualitative Reasoning

Beyond the quantitative metrics, the reasoning provided by the LLMs offers insight into how they approached the CFG reconstruction task. The explanations exhibit several patterns that help explain the characteristics observed in the quantitative evaluation.

The LLMs demonstrated an understanding of high-level executable structure by correctly identifying function boundaries using common binary analysis patterns such as function prologues, call targets, ELF startup routines, and PLT stubs. This behavior remained largely consistent for all optimization levels, suggesting that compiler optimizations have only a limited influence on the models' ability to recognize the overall patterns of disassemblies.

Similarly, the reasoning frequently described basic block construction using explicit control-flow instructions such as conditional branches, unconditional jumps, return instructions, and fall-through execution. This indicates that the models generally understand the formal concept of a basic block and are capable of applying it during CFG reconstruction.

A recurring limitation is indirect control flow. On numerous occasions, the models reported that indirect jumps, jump tables, computed branches, or dynamically determined branch targets could not be resolved with sufficient confidence. Rather than introducing speculative control-flow edges, the models generally omitted uncertain transitions. While this conservative behavior avoids hallucinated control flow, it also produces incomplete CFGs with fewer nodes and edges, providing a likely explanation for the differences observed throughout the quantitative evaluation.

Another observation is that the evaluated LLMs correctly followed the reconstruction requirements by restricting their output to intraprocedural control flow. Function calls remained within the current basic block, interprocedural call

and return edges were excluded, and external library functions were treated as leaf nodes without reconstructing their internal control flow. This demonstrates that the models generally understood the reconstruction task and adhered to the specified constraints instead of generating arbitrary graph structures.

Overall, the qualitative reasoning demonstrates that the evaluated LLMs generally understand the fundamental principles of CFG construction, including function boundary identification, basic block formation, and intraprocedural control flow. However, they struggle with indirect control flow, dynamic dispatch, and other binary-specific constructs, resulting in structurally incomplete CFGs despite often providing coherent and technically sound explanations of their reconstruction process. These observations complement the quantitative evaluation by explaining why LLM-generated CFGs contain fewer nodes, edges, and control-flow structures than those produced by traditional binary analysis frameworks.

6.5 Limitations & Future Work

This study has several limitations that should be considered when interpreting the results.

Dataset.

The evaluation was conducted on a dataset of ten open-source C programs. While these programs span multiple application domains, they represent small- to medium-sized programs. Larger, industrial-grade software systems typically involve complex build environments, extensive external library dependencies, and substantially more intricate control-flow structures. Consequently, these findings may not fully generalize to large-scale software development environments. Future research should evaluate LLM-based CFG reconstruction on more diverse and large-scale codebases to validate the scalability of these results.

Evaluated LLMs.

This study evaluated three state-of-the-art models: Grok, Gemini, and DeepSeek. Although these represent current leading general-purpose architectures, the performance of other proprietary or open-source models may differ significantly. As LLM architectures continue to evolve, their capacity for structural reasoning and control-flow recovery will likely advance, potentially altering the performance hierarchy observed here.

Program Representation.

The evaluated models received disassembly generated from compiled binaries rather than the raw executable files themselves. While assembly listings are the primary format for human-centric reverse engineering, they offer a different information density than direct binary analysis. Thus, the observed results reflect CFG reconstruction from textual assembly rather than direct binary ingestion. Future studies could investigate whether providing raw binary data or supplemental metadata enhances reconstruction accuracy.

Compiler and Analysis Configuration.

All programs were compiled using the LLVM/Clang toolchain within a controlled pipeline. Because compiler optimizations can significantly alter binary layout, different compiler infrastructures, architectures, or non-standard optimization levels might produce varied control-flow structures that influence LLM performance. Future work should assess the impact of cross-architecture and cross-compiler variability on the structural correspondence between generated and traditional CFGs.

Reference Representations.

This study evaluates LLM-generated CFGs against established traditional analysis techniques rather than an absolute ground truth. However, as different analysis techniques recover varying levels of control-flow detail, this methodology should be interpreted as a reference-based evaluation rather than an assessment against a singular, definitive ground truth.

Prompt Design.

To ensure consistency, a standardized prompt was used across all models. It is possible that iterative prompting, Chain-of-Thought (CoT) techniques, or specialized task-specific prompting strategies could improve reconstruction fidelity. Further investigation into optimal prompting for structural reverse engineering is warranted.

Evaluation Metrics.

The evaluation prioritized structural graph metrics, including size, cyclomatic complexity, and connectivity. While these metrics effectively quantify the structural gap between representations, they do not measure semantic equivalence or execution-level correctness. Future research could complement this framework by incorporating path-based similarity measures or dynamic execution analysis to provide a more holistic evaluation of LLM-based decompilation.

LLM Non-Determinism.

Large Language Models are non-deterministic, and each evaluated model was executed only once using a standardized prompt. Consequently, the reported CFGs represent a single sample of the models' behavior rather than their complete output distribution. Future work should evaluate multiple generations per benchmark and analyze the variability of the reconstructed CFGs to determine the reliability of LLM-based CFG reconstruction.

7 Related Work

7.1 Traditional Reverse Engineering

Traditionally, control-flow graph (CFG) reconstruction has relied on static, dynamic, and hybrid binary analysis techniques.

Dynamic analysis reconstructs control flow by observing actual program execution. Because execution reveals only the paths exercised during a particular run, dynamic techniques are commonly used to complement static analysis, potentially improving control-flow graph completeness. Such approaches may also support capabilities including operating system

signals, shared code regions, multithreaded execution, exact profiling, and incremental refinement.[2]

Hybrid techniques combine static analysis with runtime exploration to enhance the completeness of reconstructed CFGs. A primary challenge in this domain is the resolution of indirect jumps, which are often difficult to recover accurately through static analysis alone. By incorporating directed gray-box fuzzing and iteratively integrating newly discovered control-flow relationships into the static graph, hybrid approaches achieve higher completeness than traditional static reconstruction methods.[26]

Static analysis reconstructs control flow without executing the program. To increase precision, abstract interpretation can be integrated with data-flow analysis and partial CFGs to compute sound overapproximations of executable control flow. This enables reconstruction without relying on compiler-specific assumptions or debugging information.[11]

7.2 Large Language Models for Reverse Engineering

Large language models (LLMs) have recently been applied to various reverse engineering tasks, including program understanding, CFG generation, and low-level code analysis.

Current research in this field remains emergent. Existing surveys identify challenges related to reproducibility, evaluation methodologies, and the lack of standardized benchmarks, noting that while much of the existing research focuses on improving model performance, areas such as robustness, interpretability, and comprehensive evaluation remain comparatively underexplored.[7]

Specific efforts to utilize LLMs for CFG generation have moved beyond traditional bytecode- or AST-based techniques. By decomposing CFG generation into distinct reasoning stages—such as structure extraction, nested code block analysis, and graph fusion—these approaches improve node and edge coverage, particularly for incomplete or erroneous code.[8] However, the field acknowledges an ongoing need to evaluate these approaches across diverse programming languages and newer model architectures to assess broader generalizability.

Most recently, studies have evaluated the ability of LLMs to interpret compiler intermediate representations (IR). While recent models successfully parse IR syntax and identify high-level program structures, they struggle with instruction-level reasoning, loop handling, and execution analysis.[10] These findings suggest that future enhancements in control-flow awareness, IR-specific training, and graph-based architectures are necessary. Crucially, these evaluations focus on IRs rather than the distinct reverse engineering scenario in which only compiled binaries or disassembly are available, leaving a gap for the study presented here.

8 Conclusion

We evaluated the ability of LLMs (Gemini, DeepSeek, and Grok) reconstruct CFGs from binary disassembly and compared their performance against traditional approaches (LLVM, Ghidra, and Rizin).

The quantitative evaluation demonstrated a clear gap between traditionally-generated and LLM-generated CFGs. Traditional approaches consistently reconstructed larger and more complete CFGs, while the LLMs produced smaller and more abstract representations. Among the evaluated LLMs, Gemini generally produced the richest CFGs, followed by DeepSeek, whereas Grok consistently generated the most compact graphs. Although graph density exhibited a different ordering, this was primarily a consequence of graph size rather than improved reconstruction quality.

The evaluation further showed that compiler optimizations have only a limited influence on the relative behavior of both traditional approaches and LLMs. The qualitative analysis further demonstrated that the LLMs generally understand fundamental CFG concepts, including function boundary identification, basic block construction, and intraprocedural control flow. However, they consistently struggle with indirect control flow, dynamic dispatch, and other binary-specific constructs, resulting in structurally incomplete CFGs.

Overall, the results indicate that current LLMs are capable of supporting high-level program understanding but do not yet provide sufficiently accurate CFG reconstruction to replace traditional binary analysis in reverse engineering. At present, they are better viewed as complementary assistants that can aid reverse engineers rather than as standalone CFG reconstruction tools.

As LLMs continue to improve in low-level binary reasoning, future work can build upon our methodology to evaluate whether emerging models narrow the observed gap and thereby increase the applicability of LLMs within reverse engineering.

References

[1] Frances E. Allen. 1970. Control flow analysis. In *Proceedings of a Symposium on Compiler Optimization* (Urbana-Champaign, Illinois). Association for Computing Machinery, New York, NY, USA, 1–19. doi:10.1145/800028.808479

[2] Andrei Rimsa Álvares. 2020. Practical dynamic reconstruction of control flow graphs. (2020).

[3] DeepSeek-AI. 2024. DeepSeek-V3 Technical Report. arXiv:2412.19437 [cs.CL] https://arxiv.org/abs/2412.19437

[4] Christof Ebert, James Cain, Giuliano Antoniol, Steve Counsell, and Phillip Laplante. 2016. Cyclomatic Complexity. *IEEE Software* 33, 6 (2016), 27–29. doi:10.1109/MS.2016.147

[5] Google DeepMind. 2026. Gemini 3.5 Flash Model Card. https://deepmind.google/models/model-cards/gemini-3-5-flash/. Published May 19, 2026. Accessed: July 20, 2026.

[6] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. 2008. Exploring Network Structure, Dynamics, and Function using NetworkX. *Python in Science Conference* (2008). doi:10.25080/TCWV9851

[7] Xinyu Hu, Zhiwei Fu, Shaocong Xie, Steven H. H. Ding, and Philippe Charland. 2025. SoK: Potentials and Challenges of Large Language Models for Reverse Engineering. arXiv:2509.21821 [cs.CR] https://arxiv.org/abs/2509.21821

[8] Qing Huang, Zhou Zou, Zhenchang Xing, Zhenkang Zuo, Xiwei Xu, and Qinghua Lu. 2023. AI Chain on Large Language Model

for Unsupervised Control Flow Graph Generation for Statically-Typed Partial Code. arXiv:2306.00757 [cs.SE] https://arxiv.org/abs/2306.00757

[9] Haseeb Javed, Farman Ali, Babar Shah, and Daehan Kwak. 2025. Binary Code Analysis for Cybersecurity: A Systematic Review of Forensic Techniques in Vulnerability Detection and Anti-Evasion Strategies. *IEEE Access* 13 (2025), 167139–167164. doi:10.1109/ACCESS.2025.3610616

[10] Hailong Jiang, Jianfeng Zhu, Yao Wan, Bo Fang, Hongyu Zhang, Ruoming Jin, and Qiang Guan. 2025. Can Large Language Models Understand Intermediate Representations in Compilers? arXiv:2502.06854 [cs.LG] https://arxiv.org/abs/2502.06854

[11] Johannes Kinder, Florian Zuleger, and Helmut Veith. 2009. An abstract interpretation-based framework for control flow reconstruction from binaries. In *International Workshop on Verification, Model Checking, and Abstract Interpretation*. Springer, 214–228.

[12] C. Lattner and V. Adve. 2004. LLVM: a compilation framework for lifelong program analysis & transformation. In *International Symposium on Code Generation and Optimization, 2004. CGO 2004.* 75–86. doi:10.1109/CGO.2004.1281665

[13] Shervin Minaee, Tomas Mikolov, Narjes Nikzad, Meysam Chenaghlu, Richard Socher, Xavier Amatriain, and Jianfeng Gao. 2025. Large Language Models: A Survey. arXiv:2402.06196 [cs.CL] https://arxiv.org/abs/2402.06196

[14] National Security Agency. 2019. Ghidra: A Software Reverse Engineering Framework. https://github.com/NationalSecurityAgency/ghidra. Accessed: 2026-06-24.

[15] Minh Hai Nguyen, Thien Binh Nguyen, Thanh Tho Quan, and Mizuhito Ogawa. 2013. A Hybrid Approach for Control Flow Graph Construction from Binary Code. In *2013 20th Asia-Pacific Software Engineering Conference (APSEC)*, Vol. 2. 159–164. doi:10.1109/APSEC.2013.132

[16] Chengbin Pang, Tiantai Zhang, Ruotong Yu, Bing Mao, and Jun Xu. 2022. Ground Truth for Binary Disassembly is Not Easy. In *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, Boston, MA, 2479–2495. https://www.usenix.org/conference/usenixsecurity22/presentation/pang-chengbin

[17] Hammond Pearce, Benjamin Tan, Prashanth Krishnamurthy, Farshad Khorrami, Ramesh Karri, and Brendan Dolan-Gavitt. 2022. Pop Quiz! Can a Large Language Model Help With Reverse Engineering? arXiv:2202.01142 [cs.SE] https://arxiv.org/abs/2202.01142

[18] Rizin Organization. [n. d.]. *The Rizin Handbook: A Guide to Reverse Engineering with the Rizin Framework*. https://book.rizin.re/

[19] Huanyao Rong, Yue Duan, Hang Zhang, XiaoFeng Wang, Hongbo Chen, Shengchen Duan, and Shen Wang. 2024. Disassembling Obfuscated Executables with LLM. arXiv:2407.08924 [cs.CR] https://arxiv.org/abs/2407.08924

[20] Yan Shoshitaishvili, Ruoyu Wang, Audrey Dutcher, Christopher Kruegel, and Giovanni Vigna. 2017. angr: A Powerful and User-friendly Binary Analysis Platform. *en. In* (2017).

[21] Yulei Sui and Jingling Xue. 2016. SVF: interprocedural static value-flow analysis in LLVM. In *Proceedings of the 25th international conference on compiler construction*. 265–266.

[22] Hanzhuo Tan, Weihao Li, Xiaolong Tian, Siyi Wang, Jiaming Liu, Jing Li, and Yuqun Zhang. 2025. SK2Decompile: LLM-based Two-Phase Binary Decompilation from Skeleton to Skin. *arXiv preprint arXiv:2509.22114* (2025).

[23] USENIX. 2026. USENIX Security Ethics Guidelines. https://www.usenix.org/conference/usenixsecurity26/call-for-papers#ethics

[24] USENIX Security Artifact Evaluation Committee. 2025. Security Research Artifacts. https://secartifacts.github.io

[25] xAI. 2025. Grok 4. https://x.ai/news/grok-4. Accessed: July 11, 2026.

[26] Kailong Zhu, Yuliang Lu, Hui Huang, Lu Yu, and Jiazhen Zhao. 2021. Constructing more complete control flow graphs utilizing directed gray-box fuzzing. *Applied Sciences* 11, 3 (2021), 1351.

A Prompt

```
You are given a reverse engineering task and a program representation (either a binary or its disassembly).

Your task is to reconstruct the program's function-level Control Flow Graph (CFG).

Definition

A Control Flow Graph (CFG) is an intraprocedural directed graph representing the possible execution flow within
    individual functions.

Each function has its own independent CFG.

A CFG consists of:

- Nodes representing basic blocks.
- Directed edges representing possible transfers of control between basic blocks.

A basic block is defined as a maximal linear sequence of instructions with a single entry point and a single exit
    point, containing no internal control-flow transfers or branch targets. Execution proceeds sequentially until the
    terminating instruction.
Do not split a basic block into multiple nodes unless a control-flow instruction or branch target requires a new basic
    block. Consecutive non-branching instructions must belong to the same basic block.

Function call instructions do not by themselves terminate a basic block. A call instruction remains part of the
    current basic block.
Do not create separate basic blocks solely because a function call occurs.

A basic block terminates at the first instruction that may alter control flow, including but not limited to:

- conditional branches,
- unconditional jumps,
- switch dispatches,
- indirect jumps,
- returns,
- unreachable instructions.

CFG Construction Rules

Construct only intraprocedural control flow.

If multiple functions are present:

- reconstruct the CFG of each function independently;
- include all reconstructed basic blocks in a single output graph;
- do NOT introduce edges between different functions.

Specifically:

- Do not create call edges.
- Do not create return edges between caller and callee.
- Do not model the call graph.
- Do not infer interprocedural control-flow relationships.
- Ignore library functions unless their internal control flow is explicitly present.
- Only include control-flow transitions that occur within the same function.
- When the destination of a particular branch cannot be determined with reasonable confidence, omit only that specific
    edge while continuing to reconstruct the remainder of the CFG. Do not omit entire functions or the entire CFG
    solely because some edges are uncertain.

Output Format

Return ONLY valid JSON.
```

```
Use the following schema:

{
  "cfg_confidence": 0,
  "reasoning": [
    "reason 1",
    "reason 2",
    "reason 3"
  ],
  "cfg": {
    "nodes": [
      "main::B0",
      "main::B1",
      "parse_input::B0"
    ],
    "edges": [
      ["main::B0", "main::B1"],
      ["parse_input::B0", "parse_input::B1"]
    ]
  }
}

Output Rules

- Return only valid JSON.
- Do not use Markdown.
- Do not include explanations outside the JSON.
- Use node identifiers in the format "<function_name>::B<n>", where <function_name> is the recovered function name if
    identifiable (e.g., "main", "parse_input"), or a generated identifier such as "sub_1", "sub_2", etc.
- Number basic blocks independently within each function (e.g., main::B0, main::B1, helper::B0, helper::B1).
- Each node must correspond to exactly one basic block.
- Each basic block must belong to exactly one function.
- Each edge must represent one possible intraprocedural control-flow transition.
- Do not create interprocedural edges. Every edge must connect two basic blocks belonging to the same function.
- Do not create edges representing function calls.
- Do not create edges representing returns to callers.
- cfg_confidence must be an integer between 0 and 100.
- The returned CFG should represent your best reconstruction of the program's intraprocedural control flow.
- Assume the supplied program representation is complete unless it is explicitly truncated or incomplete.
- Never return an empty CFG solely because function names are unavailable or because some control-flow cannot be
    recovered.
- Produce the most complete sound intraprocedural CFG supported by the available evidence.
- Prefer partial reconstruction over omission. Recover all confidently identifiable functions, basic blocks, and
    intraprocedural edges even if some portions of the program remain uncertain.
- An empty CFG should only be returned if the input contains no executable code or no recoverable control-flow
    information.
```

B LLM Reasoning

B.1 DeepSeek – 2048.c (2048_00.json)

Confidence: 70

Reasoning

Function boundaries were identified based on known PLT stubs, standard ELF entry patterns, and user-defined function prologues.

Control-flow instructions such as calls (jmp), conditional branches (je, jne), direct jumps (jmp), rets, and hlt were used to mark basic block termination.

Switch statements (indirect jumps) such as the jump table at 0x3010 were reconstructed conservatively; the destination for each case was traced using the base address of the jump table.

Function calls were kept within the same basic block; edges were not introduced for calls or returns.

The internal control flow of library functions (e.g., puts, printf, rand, etc.) was not modeled because their definitions are not present in the disassembly.

Some basic blocks have fall-through paths that were inferred from sequential execution where no explicit jump exists.

Unconditional hlt at 0x1135 marks the end of a basic block with no outgoing edge.

Edges from indirect jumps (e.g., jmp rax) were omitted when the destination could not be resolved, but the rest of the CFG was preserved.

The function sub_1200 includes a computed load and direct byte access but no branching; it was treated as one basic block.

Functions sub_13A0, sub_1630, sub_16D0, sub_17E0, sub_1970, sub_19E0, sub_1AA0, sub_1B00, sub_1B90, sub_1C00, sub_1C90, sub_1DA0, sub_1E10, sub_1EA0, sub_21A0, and sub_21E0 were reconstructed from their prologues and ret instructions.

No interprocedural edges were added between any functions.

Some functions (like sub_1200) were analyzed entirely without branching and considered as single-block functions.

All edges were verified to be intraprocedural.

B.2 DeepSeek – 2048.c (2048_01.json)

Confidence: 85

Reasoning

The disassembly contains multiple identifiable functions: main (0x29c0), sub_12e0 (0x12e0), sub_2590 (0x2590), sub_26a0 (0x26a0), sub_2740 (0x2740), and others.

The .text section includes the entry point and various helper functions. I have reconstructed CFGs for all major functions where control flow is discernible.

Some functions (like sub_12e0) have complex control flow with multiple branches and loops, which I have modeled based on the conditional jumps and loop patterns.

For functions with indirect jumps (e.g., switch-like structures such as in main and sub_2740), I have identified the possible targets based on the jump table or comparison patterns.

Edges are drawn strictly within each function; no interprocedural edges (calls or returns) are included.

I omitted edges for branch destinations that could not be resolved with reasonable confidence, but still included all resolved intraprocedural edges.

The confidence score reflects that while most control flow is recoverable, some indirect jumps and unreachable paths reduce certainty.

B.3 DeepSeek – 2048.c (2048_02.json)

Confidence: 85

Reasoning

Identified function boundaries based on labels and call targets from the disassembly.

Recovered control flow for functions: _init, _start, call___do_global_dtors_aux, do_global_ctors_aux, sub_1200, digit_count, sub_12e0, sub_1c10, sub_1c60, sub_1d50, sub_1e20, sub_1f90, sub_20e0, sub_2250, sub_22b0, sub_2300, sub_2580, sub_2800, sub_2820, sub_28c0, sub_2bc0, main, and _fini.

Each function's basic blocks are determined by control-flow instructions (jmp, je, jne, ja, jbe, ret, call, hlt) and branch targets.

Function calls do not terminate basic blocks; they are included in the current block.

Edges are only intraprocedural. Calls to external functions (e.g., printf, puts, srand, time) are not expanded with edges.

Interprocedural edges are excluded. There are no edges between different functions.

Some indirect jumps (e.g., call rax, jmp QWORD PTR) have undetermined destinations and are omitted as per instructions.
Switch jump table at 0x2d45 is partially resolved; only identified branches are included.
The cfg_confidence is set to 85 because the control flow of most functions is clearly recoverable, though some indirect branches and switch cases are uncertain.

B.4 DeepSeek – 2048.c (2048_O3.json)

Confidence: 75

Reasoning

The disassembly contains multiple functions identified by their entry points and linking conventions.
Function 'main' starts at address 0x2dc0 and includes the program's entry logic and main loop.
Function 'sub_12e0' starts at 0x12e0 and is called from 'main' to update a display.
Function 'sub_1fe0' starts at 0x1fe0 and appears to update state based on input characters.
Function 'sub_21e0' starts at 0x21e0 and handles a specific transformation.
Function 'sub_22e0' starts at 0x22e0 and handles another transformation.
Function 'sub_23c0' starts at 0x23c0 and handles a third transformation.
Function 'sub_2570' starts at 0x2570 and performs a check on the state.
Function 'sub_27f0' starts at 0x27f0 and initializes or randomizes state.
Function 'sub_2a70' starts at 0x2a70 and calls 'sub_27f0' twice.
Function 'sub_2a90' starts at 0x2a90 and manages terminal settings.
Function 'sub_2b30' starts at 0x2b30 and displays a debug or test output.
Function 'sub_2d90' starts at 0x2d90 and is a signal handler that exits.
Edges are constructed based on conditional jumps, unconditional jumps, and returns.
Indirect calls (e.g., via 'call rax') and some computed jumps make exact edge recovery uncertain in some places.
Where destination is certain (e.g., 'je' to a fixed address), edges are added.
Where a jump table or indirect branch is used (e.g., 'jmp [rax*4+rcx]'), only the clearly identifiable direct targets are included.
No interprocedural edges are added; calls are treated as part of the current basic block.
All identified functions and their basic blocks are included; uncertain edges are omitted.

B.5 DeepSeek – Notepad– (Notepad--_O0.json)

Confidence: 75

Reasoning

The disassembly contains multiple functions, including the main function starting at 0x4580.
For each function, basic blocks were identified by analyzing control-flow instructions (jumps, conditional jumps, calls to exit, etc.).
Intraprocedural edges were constructed based on direct jumps, conditional branches, and fall-through behavior.
Edges for indirect jumps or calls (e.g., call rax) were omitted due to unknown targets.
Edges to external functions (e.g., exit) are not represented as edges to other basic blocks.
Some functions like __cxa_finalize@plt are not modeled as they are library stubs without internal control flow.
The CFG is a partial reconstruction; some indirect control flow could not be resolved with high confidence.

B.6 DeepSeek – Notepad– (Notepad--_O1.json)

Confidence: 75

Reasoning

Function boundaries recovered from disassembly: start and end addresses of each function are determined by labels and section layout. Functions identified include: sub_1000 (init), sub_1020 (plt), sub_1240, sub_1250, sub_12f0, sub_1330, sub_1340, sub_1400, sub_1440, sub_1460, sub_14f0, sub_1510, sub_1690, sub_1730, sub_18c0, sub_1980, sub_1a00, sub_1a70, sub_1ad0, sub_1b20, sub_1b70, sub_1c50, sub_1d40, sub_2370, sub_2490, sub_2630, sub_2710, sub_2830, sub_2930, sub_2950, sub_2960, sub_2970, sub_2990, sub_29d0, sub_2a00, sub_2ad0, sub_2b10, sub_2b40, sub_2c60, sub_2c80, sub_2d20, sub_2d90, sub_2ed0, sub_2f30, sub_3060, sub_31c0, sub_3410, sub_3470, sub_3520, sub_4280, sub_4330, sub_43a0, sub_4480, sub_4650, sub_46c0, sub_49b0, sub_4b20, sub_4b60, sub_4ba0, sub_4bd0, and sub_4ec0.

For each function, basic blocks are determined by scanning instructions for control-flow instructions (jmp, je, jne, jg, jle, jl, jge, ja, jbe, jae, jb, ret, hlt, etc.) and target labels (e.g., address references used as jump targets). Consecutive non-branching instructions are grouped into the same block.

Edges are created for each branch: for unconditional jumps, add one edge; for conditional jumps, add two edges (taken and fall-through) when the target can be determined. If a target is unknown or beyond the function boundary, the edge is omitted. For indirect jumps (e.g., jmp rax), edges are omitted because the destination is not statically determinable.

Unreachable instructions (e.g., hlt after a call) terminate a basic block without outgoing edges.

Functions that contain only calls and no intraprocedural control-flow (e.g., .plt stubs) are modeled as single basic blocks with no outgoing edges, since the control flow goes to the callee (interprocedural edge omitted).

Some functions have complex control flow with multiple branches, loops, and nested conditionals; these are reconstructed to the best ability. For uncertain indirect branches, edges are omitted but the rest of the CFG is preserved.

The overall confidence is moderate (75) because some indirect jumps and calls with dynamic targets cannot be resolved, and some function boundaries are inferred from the disassembly without debug symbols.

B.7 DeepSeek – Notepad– (`Notepad--_O2.json`)

Confidence: 75

Reasoning

The disassembly contains multiple functions including the entry point, initialization routines, and various helper functions.

The entry point at 0x1260 calls __libc_start_main, which invokes main-like functionality at 0x4dc0.

Basic blocks are identified based on control flow instructions (conditional/unconditional jumps, calls, returns).

Function boundaries are determined from the disassembly layout and calling conventions.

The main function at 0x4dc0 contains a loop that reads input and processes commands.

Several helper functions are identified: 0x13f0 (write_buffer), 0x14f0 (setup_terminal), 0x1500 (init_editor), 0x1670 (add_buffer), 0x1700 (load_file), 0x1890 (free_all), 0x1950 (get_word), 0x19d0 (set_word), 0x1a40 (copy_string), 0x1aa0 (init_string), 0x1ae0 (delete_buffer), 0x1b40 (insert_buffer), 0x1c20 (remove_buffer), 0x1d10 (insert_char), 0x22d0 (process_command), 0x23f0 (undo), 0x2570 (display_status), 0x2650 (print_status), 0x2770 (handle_escape), 0x2850 (scroll_up), 0x2870 (scroll_down), 0x2880 (scroll_right), 0x2890 (scroll_left), 0x28b0 (scroll_home), 0x28f0 (scroll_end), 0x2920 (insert_string), 0x29f0 (delete_char), 0x2ac0 (delete_backward), 0x2af0 (insert_char_at), 0x2c10 (insert_space), 0x2c40 (delete_char_at), 0x2ce0 (delete_word), 0x2d50 (enter_command_mode), 0x2e90 (print_message), 0x2ef0 (read_escape_sequence), 0x3020 (handle_key), 0x3190 (read_command), 0x33e0 (check_interrupt), 0x3440 (print_prompt), 0x34f0 (process_buffer), 0x44b0 (analyze_buffer), 0x4560 (find_pattern), 0x45d0 (replace_pattern), 0x46b0 (expand_macros), 0x4850 (remove_markers), 0x48c0 (load_config), 0x4ba0 (save_file), 0x4d10 (file_size), 0x4d50 (check_word_boundary), 0x4d90 (strip_whitespace), and 0x4dc0 (main).

Edges are established based on conditional and unconditional jumps within each function.

Some indirect jumps (e.g., jump tables) cannot be fully resolved, so those specific edges are omitted while the rest of the CFG is preserved.

Interprocedural edges (calls and returns) are excluded per the instructions.

B.8 DeepSeek – Notepad– (`Notepad--_O3.json`)

Confidence: 70

Reasoning

The disassembly contains numerous functions, including _init, .plt stubs, .text, and .fini sections.

Functions were identified based on entry points, call/ret patterns, and prologue/epilogue sequences.

Unnamed functions in .text were assigned generated names (sub_1260, sub_1350, sub_13f0, etc.) based on their starting addresses.

Basic blocks were determined by identifying terminating instructions (jmp, je, jne, jg, jle, ret, etc.) and branch targets.

Indirect jumps (e.g., call rax, jmp rax) were identified but their destinations could not be resolved; edges to unknown targets were omitted.

Function calls (e.g., call ¡plt¿ entries) were treated as part of the basic block and not as control-flow terminators.

Some conditional branches have computed targets or rely on external data; only edges with resolvable destinations were included.

The CFG is partial but captures the majority of intraprocedural control flow for each identified function.

B.9 DeepSeek – Tinyhttpd (`Tinyhttpd_O0.json`)

Confidence: 80

Reasoning

The disassembly contains multiple functions: .init, .plt, .plt.got, .text, and .fini sections. The main executable code is in .text.

Identified functions: sub_1390, sub_1840, sub_1930, sub_1bc0, sub_1ea0, sub_1f70, sub_2380, sub_2470, sub_2510, sub_2670, sub_2690, sub_27f0, sub_2920 (likely main).

sub_1390 is a large function that handles client requests, parsing input and performing actions based on the request type. It has multiple branches.

sub_1840 is a recv-line function that reads a line from a socket.

sub_1930 and sub_1bc0 are error/send functions that output messages.

sub_1ea0 and sub_1f70 handle file operations and CGI execution.

sub_2380, sub_2470, sub_2510, sub_2670, sub_2690 are helper functions for sending error messages or file contents.

sub_27f0 sets up a listening socket.

sub_2920 appears to be the main function: it creates a socket, listens, accepts connections, and spawns threads.

For each function, I constructed basic blocks based on control-flow instructions: conditional jumps (je, jne, jge, jle), unconditional jumps (jmp), ret, and function calls that might alter flow (call followed by test, but calls are not terminating).

Some indirect jumps (e.g., call rax, jmp rax) are not resolved, so I omitted edges from those instructions.

I included all identifiable branches where the destination is known (e.g., direct jumps to labels).

The CFG includes all functions with their internal basic blocks and edges. Intraprocedural edges are drawn from each block's terminating instruction to its possible successors.

Edges are only within the same function. No cross-function edges are included.

Confidence is set to 80 because some indirect control flows and library call targets are not fully resolved, but the main flow is clear.

B.10 DeepSeek – Tinyhttpd (`Tinyhttpd_O1.json`)

Confidence: 85

Reasoning

The disassembly contains multiple functions including main (0x24f0), a thread handler (0x1330), and various subroutines.

Function boundaries were identified based on call targets, entry points, and return instructions.

Basic blocks were determined by analyzing control-flow instructions (jmp, jcc, call, ret) and branch targets.

Intraprocedural edges were constructed based on direct jumps, conditional branches, and fall-through paths.

Indirect jumps and calls to library functions were not resolved with certainty; however, direct intraprocedural transitions were recovered.

CFG confidence is set to 85 due to the presence of indirect calls and some unresolved branch destinations.

B.11 DeepSeek – Tinyhttpd (`Tinyhttpd_O2.json`)

Confidence: 70

Reasoning

The disassembly contains multiple functions, including what appears to be a main-like function at 0x28d0 that sets up a socket, accepts connections, and creates threads.

Function boundaries are not explicitly marked by symbols, but are inferred from entry points, call patterns, and the typical structure of the code.

Many functions rely on dynamic library calls (e.g., recv, send, printf) and process data from network sockets.

Some control flow is challenging to recover due to indirect jumps and incomplete information about some branches.

I have reconstructed intraprocedural CFGs for each identifiable function, using inferred names such as 'main', 'handle_connection', 'read_until_newline', etc.

Edges are based on explicit jumps, conditional branches, calls to exit or perror, and fall-through paths.

Edges that are uncertain (e.g., due to missing targets or indirect jumps) are omitted rather than guessed.

Function calls are not represented as edges, in accordance with the instructions.

B.12 DeepSeek – Tinyhttpd (`Tinyhttpd_O3.json`)

Confidence: 70

Reasoning

The disassembly contains multiple functions, including what appears to be a main-like function at 0x28d0 that sets up a socket, accepts connections, and creates threads.

Function boundaries are not explicitly marked by symbols, but are inferred from entry points, call patterns, and the typical structure of the code.

Many functions rely on dynamic library calls (e.g., recv, send, printf) and process data from network sockets.

Some control flow is challenging to recover due to indirect jumps and incomplete information about some branches.

I have reconstructed intraprocedural CFGs for each identifiable function, using inferred names such as 'main', 'handle_connection', 'read_until_newline', etc.

Edges are based on explicit jumps, conditional branches, calls to exit or perror, and fall-through paths.

Edges that are uncertain (e.g., due to missing targets or indirect jumps) are omitted rather than guessed.

Function calls are not represented as edges, in accordance with the instructions.

B.13 DeepSeek – bzip2 (`bzip2_O0.json`)

Confidence: 60

Reasoning

Disassembly contains many functions, including standard library PLT stubs and an extensive .text section with numerous user-defined functions.

The function at address 0x22a0 is the entry point (_start), and it calls 0x11030 (which appears to be main) via a GOT entry.

Many functions are present, such as sub_2390, sub_25c0, sub_2e30, sub_3a00, sub_3990, sub_42d0, sub_4590, sub_4650, sub_4700, sub_4b90, sub_8de0, sub_8e50, sub_dcb0, sub_dd60, sub_df60, sub_f650, sub_f760, sub_f9b0, sub_fc20, sub_10ac0, sub_10df0, sub_10e60, and many more.

The disassembly is extensive and includes many control-flow instructions (conditional jumps, unconditional jumps, calls, returns), making full manual reconstruction impractical.

All functions can be identified, and for many functions, the basic blocks can be determined by identifying branch targets and fall-through instructions.

However, due to the sheer size and complexity of the code, a complete and accurate CFG reconstruction for every function would require a sophisticated decompiler and significant manual effort.

The provided confidence score reflects that while the structure is recoverable, the full CFG is not manually reconstructed in this answer.

The output shows a representative structure for the main function based on the disassembly, but many other functions are present but not fully expanded.

B.14 DeepSeek – bzip2 (`bzip2_O1.json`)

Confidence: 60

Reasoning

The disassembly contains many functions, but many are unnamed library functions or PLT stubs. I have identified several probable user-defined functions based on their structure and address ranges.

For the sub_22c0 function, I have identified a clear control flow pattern: it starts with a conditional jump based on ebx, and then always calls exit.

For the sub_2320 function, I have identified several conditional branches and calls to other functions, but the exact internal logic is complex.

For the sub_2620 function, I have reconstructed a large CFG with many basic blocks based on the conditional branches, jumps, and loops present in the disassembly.

Many functions, like sub_4810, sub_7fc0, sub_ab10, etc., show clear control-flow structures including conditional jumps and function calls, which have been modeled as basic blocks and intraprocedural edges.

For functions with indirect jumps or switch-like structures (e.g., sub_7fc0 uses a jump table), I have omitted edges where the destination is uncertain but still included the basic block structure.

The CFG does not include interprocedural edges (e.g., no call edges or return edges between functions), only intraprocedural control flow within each function.

The PLT stubs and library functions (e.g., getenv, malloc) are not analyzed as their internal control flow is not present in the disassembly.

The reasoning list contains multiple concise justifications for the reconstruction choices.

The confidence score is 60, reflecting a partial but sound reconstruction where some edges are uncertain but the main structure is recoverable.

B.15 DeepSeek – bzip2 (`bzip2_O2.json`)

Confidence: 70

Reasoning

The disassembly contains multiple functions and subroutines.

The main function entry point is identified at address 0x2530.

Control flow within functions is followed using conditional and unconditional jumps.

Some indirect jumps and calls to external functions make certain edges uncertain.

Edges are reconstructed based on the most likely execution paths from the disassembly.

The confidence score reflects the presence of indirect branches and unresolved targets.

B.16 DeepSeek – bzip2 (`bzip2_O3.json`)

Confidence: 80

Reasoning

The provided disassembly contains multiple functions. I have identified and named the entry point function ‘main‘ (at address 0x2530) and several other functions based on their roles.

I have traced the control flow within each function to identify basic blocks. Basic blocks are terminated by control-flow instructions such as conditional jumps (‘je‘, ‘jne‘, ‘jg‘, ‘jl‘, etc.), unconditional jumps (‘jmp‘), returns (‘ret‘), and indirect jumps (‘jmp rax‘).

For the ‘main‘ function, I have reconstructed the CFG including the initialization, signal setup, environment variable parsing, argument handling, and the final call to ‘13520‘, ‘142e0‘, or ‘15250‘ based on command-line options.

For the ‘sub_2620‘ function, I have identified the main loop and its branches based on the value of ‘r13d‘ and the control flow in the large loop.

For other identified functions like ‘sub_5be0‘, ‘sub_aa50‘, ‘sub_f4a0‘, ‘sub_ff40‘, ‘sub_10020‘, ‘sub_10eb0‘, ‘sub_10f30‘, ‘sub_11060‘, ‘sub_11240‘, ‘sub_115a0‘, ‘sub_11880‘, ‘sub_11b30‘, ‘sub_12020‘, ‘sub_12310‘, ‘sub_12350‘, ‘sub_12390‘, ‘sub_12560‘, ‘sub_13520‘, ‘sub_142e0‘, ‘sub_15250‘, and ‘sub_15ac0‘, I have created nodes and edges based on their internal control-flow instructions.

Some edges, particularly those involving indirect jumps (‘jmp rax‘) and computed branches (e.g., jump tables), are challenging to resolve statically. I have omitted edges where the destination could not be determined with reasonable confidence.

I have not created edges for function calls (e.g., ‘call 2180‘), as per the instructions. The control flow remains within the current basic block after a call.

The ‘cfg_confidence‘ is set to 80, reflecting a high but not perfect degree of confidence. The main control flow and structure are well-recovered, but some indirect and dynamic branches are omitted.

B.17 DeepSeek – fe (`fe_O0.json`)

Confidence: 70

Reasoning

The disassembly contains many functions, but function boundaries are not explicitly marked with labels. Most code appears to be part of a single large function or multiple functions without clear starting points.

Control flow instructions such as conditional jumps (je, jne, jmp), calls, and ret are present, but many branch targets are computed dynamically or reference unknown offsets.

Basic blocks can be recovered at the instruction level, but function boundaries are uncertain because the disassembly lacks function entry labels.

Library functions are called but their internal control flow is not present, so they are ignored.

I will treat the entire .text section as a single function named 'sub_main' because the entry point at 0x1120 appears to be the program entry and no other clear function entry points are identifiable.

Within this single function, I identified sequential instruction sequences separated by control-flow instructions (jmp, je, jne, jmp rax, ret, etc.).

Some indirect jumps (e.g., jmp rax) have unknown targets, so those edges are omitted, but the blocks themselves are included.

The CFG is constructed by scanning the disassembly and splitting at every control-flow instruction that can alter the linear execution flow.

B.18 DeepSeek – fe (`fe_O1.json`)

Confidence: 75

Reasoning

The disassembly contains multiple functions identified by entry points and control flow patterns.

Functions identified: sub_1120 (startup), sub_1150, sub_1180, sub_11b0, sub_11f0, sub_1230, sub_1240, sub_1250, sub_1320, sub_1360, sub_13b0, sub_13d0, sub_13e0, sub_1410, sub_1420, sub_1430, sub_14b0, sub_1540, sub_15e0, sub_1670, sub_1750, sub_1820, sub_1a00, sub_1a90, sub_1b20, sub_1bf0, sub_1c60, sub_1cc0, sub_1d30, sub_2020, sub_2040, sub_2050, sub_2070, sub_20d0, sub_2130, sub_2140, sub_2170, sub_27e0, sub_2810, sub_2830, sub_2840, sub_35c0, sub_3760, sub_3780, sub_3860, sub_39c0, sub_3a90, sub_3b90, sub_3c20.

Each function's CFG was reconstructed by identifying basic blocks delimited by control-flow instructions (conditional jumps, unconditional jumps, calls to __cxa_finalize, ret, etc.) and branch targets.

For many functions, indirect jumps (e.g., call rax, jmp rax, jmp QWORD PTR) make precise destination recovery uncertain; only direct intraprocedural edges are included.

Edges that are direct and confidently determined are included; indirect edges are omitted where target cannot be resolved.

Function call instructions (e.g., call 10c0 ¡fprintf@plt¿) are treated as part of the current basic block and do not terminate the block.

The CFG includes all functions with at least one basic block and confidently identifiable control flow.

Some functions (e.g., sub_1120, sub_1150) have incomplete CFG due to indirect jumps or unresolved targets, but basic blocks are still reconstructed.

Edges between functions (interprocedural) are omitted per instructions.

B.19 DeepSeek – fe (`fe_O2.json`)

Confidence: 80

Reasoning

The disassembly contains both .init and .fini sections, but these are not user functions and contain minimal control flow.

The .text section contains multiple identifiable functions. The function at 0x1120 appears to be _start, which sets up the runtime and calls __libc_start_main.

The function at 0x1250 appears to be an error reporting function, as it calls fprintf with a format string and longjmp.

The function at 0x3730 (likely a custom initialization function) constructs a data structure, initializes arrays, and contains loops for building linked lists.

The function at 0x38e0 appears to be a cleanup/teardown function that iterates through internal data structures.

The function at 0x39c0 appears to be main, based on the argument handling and file operations.

Several other functions (sub_14b0, sub_1540, sub_1670, sub_1750, sub_1820, sub_1d60, sub_22f0, sub_2a00) contain complex control flow with conditional branches, loops, and indirect calls.

Edges are constructed based on conditional jumps, unconditional jumps, and fall-through paths in the disassembly.

Some indirect jumps (e.g., jmp rax) and computed jumps (jmp [rip+...]) cannot be resolved with certainty, so edges for those destinations are omitted.

Function calls (call instructions) do not terminate basic blocks and are not represented as CFG edges.

Library functions (printf, fprintf, strlen, etc.) are not expanded; their calls remain inside the basic blocks of the calling functions.

The CFG for each function is constructed independently, with no edges between different functions.

B.20 DeepSeek – fe (`fe_O3.json`)

Confidence: 70

Reasoning

Function boundaries are identified by call/ret patterns and PLT stubs.

Control-flow instructions are identified: conditional jumps, unconditional jumps, calls, returns.

Basic blocks are split at branch targets and terminating instructions.

Library functions (plt) are not analyzed internally.

Indirect jumps and calls are not resolved, so some edges are omitted.

The main executable functions are recovered: init, fini, and a large body starting at 0x1120.

Internal function names are not present; generated names like sub_1120 are used.

CFG is constructed for each distinct function based on its local control flow.

B.21 DeepSeek – kilo (`kilo_O0.json`)

Confidence: 75

Reasoning

The disassembly contains multiple functions including main, various subroutines, and PLT stubs.

PLT stubs are library wrappers and contain only jumps to library code; their internal control flow is not part of the analyzed program and is excluded from the CFG.

Function boundaries are identified based on labels and call/ret patterns. The .init and .fini sections are excluded as they are not part of the program's main control flow.

Basic blocks are identified by sequential instructions ending at control-flow instructions such as conditional/unconditional jumps, ret, or hlt.

Some functions (e.g., sub_22a0, sub_2310) contain indirect jumps or calls via function pointers, making some edges uncertain. However, the main control flow and many branches are recoverable.

The main function (0x53d0) calls initialization and then enters an event loop. All control-flow edges within each function are intraprocedural.

B.22 DeepSeek – kilo (`kilo_O1.json`)

Confidence: 85

Reasoning

The disassembly contains multiple functions: the startup code at .text (0x2270), __libc_csu_init (0x22a0), __libc_csu_fini (0x2350), _fini (0x4e94), and several application functions.

Function boundaries were inferred from entry points and return instructions, as well as the presence of function prologues/epilogues and calls.

The functions recoverable with high confidence include: start (0x2270), __libc_csu_init (0x22a0), __libc_csu_fini (0x2350), _fini (0x4e94), and sub_2360, sub_2390, sub_23c0, sub_2490, sub_2630, sub_26f0, sub_28e0, sub_2930, sub_2970, sub_2e00, sub_2e20, sub_2ea0, sub_2fb0, sub_30e0, sub_3100, sub_31d0, sub_3290, sub_3350, sub_33c0, sub_3400, sub_34a0, sub_3590, sub_3740, sub_3870, sub_39d0, sub_3a80, sub_3ae0, sub_3af0, sub_43f0, sub_47f0, sub_49d0, sub_4bb0, sub_4bc0, sub_4c00, sub_4c80, sub_4d10.

For each function, basic blocks were determined by scanning for control-flow instructions: conditional jumps (je, jne, jg, jl, etc.), unconditional jumps (jmp), indirect jumps (jmp rax, jmp QWORD PTR), and returns (ret).

Branch target addresses were computed from relative offsets or from register values where possible. For indirect jumps where the target could not be resolved, outgoing edges were conservatively omitted for that specific jump, but the block was still included.

Function calls (call) do not terminate basic blocks; the instruction following the call remains in the same block.

Edges were added for all resolved intraprocedural transfers: fall-through to next block, conditional branches to taken and fall-through targets, unconditional jumps, and returns to the function's epilogue.

All functions and their basic blocks were included. No interprocedural edges (calls, returns) were added.

Some blocks end with ret and have no outgoing edges, which is correct for leaf functions or function exits.

The confidence is 85 because while most control flow is recoverable, a few indirect jumps and some computed branch targets (like switch tables) could not be fully resolved from the disassembly alone.

B.23 DeepSeek – kilo (`kilo_O2.json`)

Confidence: 75

Reasoning

The disassembly contains multiple functions, including main (at 0x51b0), sub_2490 (read key), sub_26f0 (ioctl), sub_2930, sub_2970, sub_2ed0, sub_3100, sub_3290, sub_3380, sub_35a0, sub_3660, sub_3710, sub_3860, sub_3950, sub_3b00, sub_3c30, sub_4020, sub_4cf0, sub_4f00, and several helper functions.

Control flow within each function is reconstructed based on conditional jumps, unconditional jumps, calls, and returns.

Function calls (e.g., call instructions) do not terminate basic blocks; edges are only drawn for intraprocedural control transfers.

Indirect jumps (e.g., jmp rax) and computed branches (e.g., switch tables) are partially resolved when targets are identifiable; uncertain edges are omitted.

Some functions, such as sub_2930 and sub_2970, are small and have straightforward linear or conditional flow.

The main function includes a loop that processes commands and calls sub_4f00, sub_4020, and other handlers.

The CFG is not fully complete for every function due to indirect jumps and switch tables where targets could not be determined with high confidence.

All basic blocks are numbered sequentially within each function, and edges are listed for intraprocedural jumps and branches.

B.24 DeepSeek – kilo (`kilo_O3.json`)

Confidence: 75

Reasoning

The disassembly contains multiple functions including main-like entry at 0x52e0, signal handlers, and various subroutines.

The entry point at 0x52e0 corresponds to the 'main' function based on typical startup behavior and argument handling.

Several functions are identifiable from their use of library calls and control flow patterns, but many are anonymous subroutines.

Function boundaries are inferred from call/ret patterns and alignment directives.

Some indirect jumps and computed branches (e.g., switch tables) are partially recoverable from jump tables in the .text section.

Edges involving indirect calls or jumps whose targets cannot be determined with confidence are omitted.

Library functions (e.g., puts, malloc) are treated as leaves; no interprocedural edges are created for calls to them.

The CFG is constructed on a per-function basis without interprocedural edges, as required.

B.25 DeepSeek – lsh (`lsh_O0.json`)

Confidence: 85

Reasoning

Function names are partially identifiable from PLT calls and context; functions include main, sub_1210 (chdir), sub_1270 (print commands), sub_1310, sub_1320 (get count), sub_1330 (execute command), sub_13e0 (dispatch), sub_1480 (read line), sub_1580 (tokenize), sub_16b0 (shell loop), and sub_1710 (entry).

Basic blocks are split at conditional jumps, unconditional jumps, returns, and indirect calls. Call instructions do not terminate blocks.

Edges are constructed based on conditional branches and fall-through paths. Indirect call edges are omitted as targets are not statically resolvable.

No interprocedural edges are created; edges only within each function.

Some uncertainty exists in the dispatch function due to indirect call through function pointer table; the fallback to execute is included, but the exact target of the indirect call is not resolved, so that edge is omitted.

B.26 DeepSeek – lsh (`lsh_O1.json`)

Confidence: 85

Reasoning

The disassembly contains multiple function bodies: the main entry point at 0x1130, and several subroutines at 0x1220, 0x1270, 0x12e0, 0x12f0, 0x1300, 0x1390, 0x1410, 0x14c0, 0x15a0, and 0x1660.

The function at 0x1130 is the startup code that calls __libc_start_main, which eventually invokes the function at 0x1660 (likely main). The CFG for 0x1130 shows no conditional branching and ends with a call, then hlt, so it is one basic block.

The function at 0x1220 (sub_1220) handles chdir and has conditional branches and a return. It has basic blocks B0 (entry and chdir), B1 (error handling), B2 (success return), B3 (failure branch).

The function at 0x1270 (sub_1270) prints a menu and returns. It has a loop, but no conditional jumps except the loop condition; it is a single basic block ending with ret.

The function at 0x12e0 (sub_12e0) is a simple return, one basic block.

The function at 0x12f0 (sub_12f0) returns a constant, one basic block.

The function at 0x1300 (sub_1300) forks and waits for child, with conditional branches for fork failure, success, and child exec. It contains basic blocks for entry, child path, wait loop, success return, and error handling.

The function at 0x1390 (sub_1390) checks a command against a table and either executes it or calls sub_1300. It has a loop for string comparison and conditional jumps to internal functions.

The function at 0x1410 (sub_1410) reads input with getc, handles memory allocation, and has loops and error handling. It has multiple basic blocks for allocation, reading, reallocation, error, and normal return.

The function at 0x14c0 (sub_14c0) tokenizes a string with strtok and builds an array, with allocation, loop, and error handling. It has basic blocks for entry, allocation success, token loop, reallocation, error, and return.

The function at 0x15a0 (sub_15a0) is the main loop that reads input, parses it, executes built-ins or external commands, and loops until exit. It has basic blocks for entry, loop, parsing, executing, freeing, and conditional loop continuation.

The function at 0x1660 is the actual main function that calls sub_15a0 and returns. It is one basic block.

All intraprocedural edges are determined from branch instructions (jcc, jmp, call, ret). Calls to library functions are not followed inside the CFG, but the call instruction remains in the basic block.

Edges for indirect jumps (e.g., jmp [rcx+rax]) are omitted when the target cannot be resolved, but the basic block still ends with that jump. The function may still be partially reconstructed.

No interprocedural edges are created; each function's CFG is independent.

B.27 DeepSeek – lsh (`lsh_O2.json`)

Confidence: 85

Reasoning

The disassembly contains multiple functions: '_init', '_start', 'deregister_tm_clones', 'register_tm_clones', '__do_global_dtors_aux', 'frame_dummy', 'cd_to_home', 'print_banner', 'main', 'fork_exec', 'builtin_handler', 'read_line', 'parse_input', 'shell_loop', and '_fini'.

For each function, basic blocks were identified based on control flow instructions (jumps, branches, calls to 'exit', returns) and branch targets.

Edges were constructed based on fall-through, conditional jumps, and unconditional jumps within the same function.

Interprocedural edges (calls and returns) were omitted; library calls like 'printf', 'puts', 'strcmp', 'perror', etc., are not expanded.

Indirect jumps (e.g., 'jmp QWORD PTR [rcx+rax*8]') in 'builtin_handler' are resolved to specific targets based on the jump table at '0x40b0'.

Uncertainty exists around the indirect jump in 'builtin_handler' if the jump table entries were not fully identifiable, but the targets were recovered from the data section references.

Some functions like '_init' and '_fini' have straightforward control flow and are included.

The 'shell_loop' function has a complex loop structure with conditional breaks; all intraprocedural edges are recovered.

Confidence is set to 85 due to indirect jump resolution being mostly certain but relying on static analysis of the jump table.

B.28 DeepSeek – lsh (`lsh_O3.json`)

Confidence: 85

Reasoning

Identified 8 functions based on entry points and code patterns: sub_1130 (start), sub_1220 (cd_handler), sub_1280 (show_info), sub_1300, sub_1310, sub_1320 (exec_cmd), sub_13b0 (cmd_dispatcher), sub_1440 (read_line), sub_14f0 (tokenize), sub_15d0 (main_loop), and sub_16b0 (main).

For sub_1130 (start): B0 ends with call and then hlt (unreachable), no outgoing edges.

For sub_1220 (cd_handler): B0 checks rdi, branches to B1 if zero; B1 calls chdir and branches on result; B2 handles error; B3 prints error and returns.

For sub_1280 (show_info): Single linear block with no branches, returns at end.

For sub_1300: Single block returning 0.

For sub_1310: Single block returning 3.

For sub_1320 (exec_cmd): B0 forks; branches to B1 (child), B2 (error), B3 (parent wait loop). B1 calls execvp, exits on failure. B3 loops on waitpid until child terminates, then returns. B2 prints error and jumps to return.

For sub_13b0 (cmd_dispatcher): B0 checks if command is NULL; B1 compares against builtins; B2, B3, B4 handle matches; B5 calls sub_1320 for unknown commands. Edges are conditional based on strcmp results.

For sub_1440 (read_line): B0 allocates buffer; B1 reads chars in loop; B2 handles newline; B3 handles EOF; B4 handles realloc failure. Edges reflect loop and exit conditions.

For sub_14f0 (tokenize): B0 allocates tokens; B1 first strtok; B2 loop over subsequent tokens; B3 handles realloc; B4 handles error and free; B5 returns token array.

For sub_15d0 (main_loop): B0 prints prompt; B1 reads line; B2 tokenizes; B3 checks if tokens empty; B4 dispatches builtins; B5 loops if command failed; B6 exits on success. Edges reflect loop and conditional branches.

Function sub_16b0 is main: calls main_loop, then returns 0.

Uncertainty in sub_1320 wait loop edge from waitpid to itself is represented with a self-loop; also the indirect jump from command dispatcher to builtin handlers is resolved via known function pointers.

Confidence lowered slightly due to indirect jump in sub_13b0 and incomplete recovery of all possible paths, but most control flow is deterministic.

B.29 DeepSeek – md5-c (`md5_O0.json`)

Confidence: 85

Reasoning

Identified functions: main, sub_1180, sub_1260, sub_1290, sub_12e0, sub_1400, sub_1590, sub_1770, sub_17e0 based on call targets and labels.

main function spans from 0x11e0 to 0x125a, with conditional branch at 0x11f6 leading to two separate paths.

sub_1180 is a loop printing characters from a buffer, with clear jump at 0x11bb to 0x1193.

sub_1260 performs a rotate-left operation and has no branches, single block.

sub_1290 initializes MD5 context with constants, no branches.

sub_12e0 updates MD5 context with input bytes, has conditional loop and inner block for processing 64-byte chunks.

sub_1400 performs MD5 compression round, has a loop with switch-like behavior based on round index.

sub_1590 finalizes MD5 hash with padding and bit length, includes loops and calls to sub_12e0 and sub_1400.

sub_1770 computes MD5 hash of a string, calls strlen, sub_12e0, and sub_1590.

sub_17e0 reads a file in 1024-byte chunks, updates MD5 context, then finalizes.

No interprocedural edges are added; edges only represent intraprocedural jumps, conditional branches, and fall-through.

Confidence is high because control flow is mostly explicit with direct jumps and conditional branches; some indirect calls (e.g., at 0x10af, 0x10df) are not modeled as they are interprocedural.

Each function's basic blocks are numbered sequentially from B0.

B.30 DeepSeek – md5-c (`md5_O1.json`)

Confidence: 85

Reasoning

The disassembly contains functions for initialization (.init), procedure linkage table (.plt), and finalization (.fini), which are not part of the main program logic. Their internal control flow is straightforward and not part of the recoverable application functions.

The .plt section contains stubs for external functions (free, putchar, fread, strlen, printf, malloc). These are library calls and their internal control flow is not present; no interprocedural edges are created from or to these stubs.

The .plt.got section contains a single stub for __cxa_finalize. No internal control flow is present.

The .text section contains several identifiable functions: the startup routine (likely _start), __do_global_dtors_aux, frame_dummy, a function at 0x1190 (call it sub_1190), the main function at 0x11e0, and several helper functions at 0x12b0, 0x12c0, 0x12e0, 0x1370, 0x14d0, 0x15c0, and 0x17c0.

The function at 0x1190 (sub_1190) is a small loop that prints 16 bytes from a buffer and a newline. It has a single basic block (B0) ending with an unconditional jump to putchar@plt (which is not part of this function's CFG). There is no conditional branch, so it terminates at the jump.

The main function at 0x11e0 has multiple conditional branches and calls. It has a call to sub_15c0, which modifies the local buffer, then prints it. After that, it returns. The function also has a path for no arguments where it reads from stdin into a buffer. The control flow includes branches and loops, which are captured in the basic blocks.

The helper function at 0x12b0 (sub_12b0) performs a rotation and returns. It has a single basic block.

The helper function at 0x12c0 (sub_12c0) initializes a structure with zeros and a constant XMM value, then returns. Single basic block.

The helper function at 0x12e0 (sub_12e0) is a complex function with loops and conditional branches for processing data. It has multiple basic blocks due to conditional jumps and loops.

The helper function at 0x1370 (sub_1370) is a hashing-like function with a loop and conditional branches. It has multiple basic blocks.

The helper function at 0x14d0 (sub_14d0) performs padding and processing with loops and branches. It has multiple basic blocks.

The helper function at 0x15c0 (sub_15c0) computes a hash (likely MD5 or similar) of a string, with a main loop and multiple branches. It has several basic blocks.

The helper function at 0x17c0 (sub_17c0) reads from a file, processes the data in chunks, and computes a hash. It has loops and branches. It calls free at the end.

For each function, basic blocks are identified by the first instruction that is a branch target or a control-flow instruction. Edges are created for conditional and unconditional jumps, and for fall-through. Indirect jumps (like call rax) are not resolved into edges because the target is unknown, but the basic block terminates with the call instruction and fall-through to the next instruction is assumed, as the call does not transfer control within the same function.

Return instructions (ret) terminate a basic block and have no outgoing edges (except to the caller, which is interprocedural and excluded).

The confidence is set to 85 because while most functions and edges are recoverable, there are indirect calls (call rax) that are not resolved, and some conditional branches may have unresolved destinations, but the overall CFG is mostly complete.

B.31 DeepSeek – md5-c (`md5_02.json`)

Confidence: 85

Reasoning

Identified functions: main (entry at 0x1290) and sub_1190 (helper printing function), plus several other subroutines.

Constructed CFG for main using its control-flow instructions: conditional jumps, unconditional jumps, calls (ignored as edges), and ret.

Constructed CFG for sub_1190 using its sequential printing calls and final unconditional jump to putchar.

Constructed CFG for sub_1300, sub_1310, sub_1330, sub_13c0, sub_1520, sub_1610, sub_1830 based on their control-flow instructions.

Edges reflect intraprocedural jumps and branches; call instructions are not represented as edges.

Some indirect jumps (e.g., call rax) and data-dependent branches are omitted where destination cannot be determined, but basic blocks remain.

All functions are identified by their starting addresses; main is identified by the entry point referenced in the startup code.

B.32 DeepSeek – md5-c (`md5_03.json`)

Confidence: 85

Reasoning

Function names are not explicitly provided, but based on the disassembly, I can identify multiple functions: the entry point at 0x10a0, a function at 0x1190 (likely a helper to print bytes), a function at 0x1290 (likely main or a top-level function), a function at 0x1330 (likely a padding/processing function), a function at 0x1430 (a cryptographic-like transform function), a function at 0x1590 (a finalization or hash function), a function at 0x1660 (a string processing function), and a function at 0x16c0 (a file reading function).

I reconstructed the CFG for each identified function independently, using basic block boundaries based on control-flow instructions such as conditional jumps (je, jne, jbe, jl), unconditional jumps (jmp), and returns (ret).

For the entry point at 0x10a0, it calls an external function via a pointer and then halts, so it has a single basic block with no internal edges except the call, which does not terminate the block. The block ends with hlt, which is considered a terminal instruction.

For the function at 0x1190, it has a series of printf calls and a final jmp to putchar, so it is a single basic block ending with an unconditional jump.

For the function at 0x1290, it has conditional branches (jl, jne, jmp) and calls to helper functions, so I split it into multiple basic blocks: B0 (prologue and conditional check), B1 (loop body), B2 (fallback path for no arguments), and B3 (final return block).

For the function at 0x1330, it has a conditional branch (je) and a loop with conditional jumps (jbe, jne, jmp), so I split it into B0 (initial setup and early return), B1 (loop body), B2 (after loop), and B3 (return block).

For the function at 0x1430, it has a loop with conditional branches (je, jne, jmp) and ends with a ret, so I split it into B0 (prologue and setup), B1 (loop body with multiple branches), B2 (update and check), and B3 (final update and return).

For the function at 0x1590, it has a single unconditional jump to another function and a ret, so it is one basic block ending with a ret.

For the function at 0x1660, it calls strlen and then calls other functions, but has no conditional branches, so it is one basic block ending with a ret.

For the function at 0x16c0, it has a loop with conditional branches (je, jne) and ends with a ret, so I split it into B0 (prologue and initial read), B1 (loop body), and B2 (finalization and return).

I omitted edges for indirect calls or jumps where the destination could not be determined (e.g., call rax at 0x1014, jmp rax at 0x10ef), but I retained all other confidently identifiable edges.

I assigned node names based on inferred function purposes: 'entry' for the startup code, 'print_bytes' for 0x1190, 'main' for 0x1290, 'pad_process' for 0x1330, 'transform' for 0x1430, 'finalize' for 0x1590, 'process_string' for 0x1660, and 'read_file' for 0x16c0.

B.33 DeepSeek – pdpmake (pdpmake_O0.json)

Confidence: 80

Reasoning

The disassembly contains multiple functions, including the entry point at 0x23a0, the main function at 0x62e0, and many helper functions.

Only intraprocedural control flow is reconstructed; edges represent jumps, conditional branches, and returns within each function.

Function calls do not terminate basic blocks; they are included as part of the current block.

Basic blocks are identified by control-flow instructions such as jmp, jcc (e.g., je, jne), and ret. Indirect jumps are omitted if the target cannot be reliably determined, but the surrounding blocks are still recovered.

The CFG for each function is constructed independently; no interprocedural edges (e.g., call/return) are included.

Some functions are not fully analyzed due to the large size and complexity of the disassembly, but the most significant functions (e.g., entry point, main, and key helper routines) are represented.

Confidence is set to 80 because while many basic blocks and edges are clear, some indirect branches and switch-like constructs may not have all edges resolved.

B.34 DeepSeek – pdpmake (pdpmake_O1.json)

Confidence: 70

Reasoning

The disassembly contains multiple functions identified by labels and function boundaries (e.g., error@@Base, main, etc.).

Control flow instructions such as conditional jumps (je, jne), unconditional jumps (jmp), calls, and returns are present and used to identify basic block boundaries.

Indirect jumps and calls (e.g., call rax, jmp QWORD PTR) introduce uncertainty, but many direct branches are recoverable.

Function call instructions (e.g., call ¡plt¿) do not terminate basic blocks and are treated as part of the current block.

Interprocedural edges (call/return) are omitted as per instructions.

Some functions (e.g., error@@Base) have multiple basic blocks with clear conditional and unconditional jumps.

The main function (starting at 0x6d50) is identified and has several basic blocks with conditional and unconditional branches.

Some indirect jumps (e.g., jmp rax, jmp QWORD PTR) are not resolved, so some edges are omitted, but the overall CFG is reconstructed as completely as possible.

All identifiable functions are included, and basic blocks are numbered sequentially within each function.

B.35 DeepSeek – pdpmake (pdpmake_O2.json)

Confidence: 70

Reasoning

Function boundaries are inferred from the disassembly, using labels such as error@@Base, main, and others.

Basic blocks are identified by sequences of instructions that are not terminated by control-flow changing instructions (jumps, branches, ret, etc.).

Edges are established based on direct jumps and conditional branches where the target address is within the same function.

Indirect jumps (e.g., call rax, jmp rax, jmp QWORD PTR) are not resolved to specific targets, so edges for those blocks may be incomplete.

Interprocedural control flow such as call and ret instructions are ignored; edges are only intraprocedural.

Some functions like __libc_csu_init and __libc_csu_fini are recognized and their internal control flow is reconstructed.

Due to the presence of many unresolved indirect jumps and dynamic calls, the CFG is partial but represents the most sound reconstruction possible.

The 'error@@Base' function has multiple sequential basic blocks with direct jumps, making its CFG relatively complete.

The 'main' function uses dynamic dispatch and indirect calls, so some edges are omitted.

Other functions like 'sub_2960', 'sub_3930', 'sub_5040', etc., are included with their internal control flow where possible.

A total of 42 functions are identified and basic blocks are numbered per function.

B.36 DeepSeek – pdpmake (pdpmake_O3.json)

Confidence: 60

Reasoning

The disassembly contains multiple functions, including error@@Base, which appears to be a custom error handling function, and main-like code starting at 0x23b0.

The primary function identified is error@@Base (starting at 0x23b0), which handles variable arguments and calls vfprintf and exit.

There are several helper functions identified such as sub_2970, sub_3430, sub_3480, sub_34A0, sub_34C0, sub_3550, sub_35F0, sub_36A0, sub_3710, sub_38B0, sub_3990, sub_3AB0, sub_5480, sub_5D90, sub_5E80, sub_68D0, sub_69A0, sub_6BD0, sub_6CE0, sub_6ED0, sub_70B0, sub_70F0, sub_7140, sub_71E0, sub_7230, sub_7370, sub_73A0, sub_8120, sub_8520, sub_8600, sub_8810, sub_8830, sub_8890, sub_9850, sub_9880, sub_9C30, sub_A330, sub_A430, sub_A4E0, sub_A510, sub_A610, sub_A830, sub_A860, sub_A920, sub_A990, and sub_AA20.

The function at 0x73A0 appears to be the main entry point (_start-like), with a call to __libc_start_main.

Control flow within error@@Base is primarily sequential with a single unconditional call to exit, making it a leaf function with no internal branches.

sub_2970 contains several conditional branches (e.g., comparisons with 0x24, 0x7b, 0x28) and loops, indicating it is a complex string processing function.

The function sub_3AB0 is large and contains many conditional jumps, loops, and calls to other functions, indicating it is a major parsing or execution function.

The function sub_73A0 contains the program's entry point and calls into other functions, but its internal control flow is limited to a few branches based on argc.

The CFG reconstruction includes all identified functions, but due to the complexity and the presence of indirect jumps (e.g., jmp rax), some edges are uncertain.

The confidence score is reduced because of the large number of unresolved indirect branches and the lack of symbol information for many functions.

B.37 DeepSeek – wak (wak_O0.json)

Confidence: 75

Reasoning

The disassembly includes multiple functions, but many are library stubs or small helper routines with no control-flow complexity.

The primary function of interest is at address 0x2480, which contains argument parsing, option handling, and calls to various subroutines. I will refer to this as 'main'.

The control flow within main includes a loop around getopt_long and several conditional branches, making it the most complex recoverable CFG.

Several helper functions are present but are either trivial (e.g., return immediately) or contain straightforward linear code without branching.

Function names could not be definitively recovered, so generated names are used for non-main functions.

Edges that involve indirect jumps or calls to unknown destinations are omitted to maintain accuracy.

Overall, the reconstruction focuses on the main function and its immediate subroutines, omitting library calls and uncertain indirect transitions.

B.38 DeepSeek – wak (wak_O1.json)

Confidence: 75

Reasoning

The disassembly includes multiple functions, but many are library stubs or small helper routines with no control-flow complexity.

The primary function of interest is at address 0x26c0, which contains argument parsing, option handling, and calls to various subroutines. I will refer to this as 'main'.

The control flow within main includes a loop around getopt_long and several conditional branches, making it the most complex recoverable CFG.

Several helper functions are present but are either trivial (e.g., return immediately) or contain straightforward linear code without branching.

Function names could not be definitively recovered, so generated names are used for non-main functions.

Edges that involve indirect jumps or calls to unknown destinations are omitted to maintain accuracy.

Overall, the reconstruction focuses on the main function and its immediate subroutines, omitting library calls and uncertain indirect transitions.

B.39 DeepSeek – wak (`wak_O2.json`)

Confidence: 60

Reasoning

The disassembly contains multiple function stubs and one large function starting at address 0x26c0, which appears to be the main program entry point.

The large function at 0x26c0 contains complex control flow with many conditional branches, jumps, and function calls.

Due to the complexity and size of the disassembly, fully recovering all basic blocks and edges for every function is not feasible with high confidence.

However, the main function's high-level structure has been partially recovered, including its initialization, option parsing, and main processing loop.

Several library functions are present in the PLT, but their internal control flow is not included as per the rules.

The confidence score reflects the partial nature of the reconstruction, especially for the main function's internal logic.

B.40 DeepSeek – wak (`wak_O3.json`)

Confidence: 40

Reasoning

The disassembly contains a large number of functions and library calls.

Function boundaries are not clearly defined in the provided disassembly, which lacks standard ELF symbols.

I can identify the likely entry point at address 0x26c0, which initializes the program and processes command-line options via getopt_long.

The entry point function has a large body of code with many conditional branches and jumps, but its exact control flow is complex and difficult to fully reconstruct without dynamic analysis.

Several other functions can be identified by their entry points and exit points, such as the functions at 0x2390, 0x2470, 0x4dc0, 0x4f40, 0x6040, 0x6830, 0x6a00, 0x71c0, 0x73f0, 0x8300, 0x8bf0, 0x90f0, 0xb610, 0xaa00, 0xaac0, 0xaf40, 0xb2e0, and 0xb470.

I am able to reconstruct basic blocks and edges for these functions based on direct branches and conditional jumps that are present in the code.

However, many control flow edges are indirect (e.g., jmp rax, jmp QWORD PTR [rip+...]) or depend on complex data structures, making them impossible to resolve statically.

I will include the confidently identified functions and their basic blocks, omitting only edges that cannot be determined with reasonable confidence.

The overall confidence is low due to the large number of unresolved indirect branches and the lack of clear function boundaries.

B.41 Gemini – 2048.c (`2048_O0.json`)

Confidence: 95

Reasoning

Analyzed the provided disassembly to identify individual function entry points and boundaries.

Mapped out the basic blocks within each function according to the rules (splitting at control-flow altering instructions like jumps, conditional branches, and returns).

Ignored interprocedural calls and modeled only intraprocedural execution paths.

Constructed sequential, conditional, and loop control structures within functions like sub_1200, sub_1360, sub_1390, sub_1630, sub_16d0, sub_17e0, sub_1970, sub_19e0, sub_1a40, sub_1aa0, sub_1b00, sub_1b90, sub_1c00, sub_1c90, sub_1da0, sub_1e10, sub_1ea0, sub_21a0, and main/sub_21e0.

B.42 Gemini – 2048.c (2048_01.json)

Confidence: 95

Reasoning

Identified sub_1200 as an independent function that formats items, containing a single basic block (B0) ending in ret at 12a5.

Identified sub_12b0 as a helper loop function with B0 from 12b0 to 12ce, B1 at 12c0 (loop body) conditionally looping back to 12c0 or returning at 12d3.

Identified sub_12e0 as a complex print/display routine. B0 goes from 12e0 to 135d. B1 starts at 1360 (loop over 4 elements) and has branches at 142b (loop back to 1360 or fall through). It contains nested logic, loops from 1475 to 1571 with branch at 1571, loop over 15c0 to 168e, and exits at 16e3.

Identified sub_16f0 as a character matching/scoring function with basic blocks handling branches at 16f4, 170f, 1717.

Identified sub_1730 as a movement or matrix transformation handler (e.g., 2048 game slide logic) containing several nested conditional blocks and loops.

Identified sub_17d0, sub_1870, sub_1940, sub_1c30, sub_1f20, sub_2200, sub_2250, sub_2290, sub_2590, sub_2680, sub_26a0, sub_2740, sub_2990 as various grid checking, transformation, and initialization procedures typical of an interactive terminal puzzle game.

Identified sub_29c0 as the main function containing command line argument validation (via strcmp chain starting at 29f0), terminal initialization via tcgetattr/tcsetattr, game loop driving input from getc, dynamic switch dispatch or if-else chain handling directions, usleep, and game ending screens.

B.43 Gemini – 2048.c (2048_02.json)

Confidence: 95

Reasoning

Analyzed the intraprocedural control flow within each function in the text section.

Functions extracted include sub_1000 (.init), sub_1110, sub_1140, sub_1170, sub_11b0, sub_11f0, sub_1200, sub_12b0, sub_12e0, sub_1c10, sub_1c60, sub_1d00, sub_1d50, sub_1e20, sub_1f8b, sub_20e0, sub_2250, sub_22b0, sub_2300, sub_2580, sub_2800, sub_2820, sub_28c0, sub_2bc0, main (sub_2bf0), and sub_3110 (.fini).

Extracted basic block headers and edges using termination points such as conditional jumps (je, jne, ja, jb, jl, etc.), unconditional jumps (jmp), and function returns (ret).

Ignored interprocedural control flow such as calls and returns between caller and callee.

B.44 Gemini – 2048.c (2048_03.json)

Confidence: 95

Reasoning

Analyzed the function at 0x12e0, which performs comprehensive game-state visualization with high register utilization (r15, r14, r12, rbx, rbp) and multiple inner loop/conditional branches printing data through printf.

Identified function starting at 0x1f90 as a helper that uses backward branching (0x1fb6 to 0x1fa0).

Identified function starting at 0x1fe0 (sub_1fe0), which contains intensive state-manipulation logic, shifts, and conditional jumps (e.g., 0x1fe6, 0x2001, 0x2008, 0x203e, 0x204a, 0x206e, 0x207a, 0x2090, 0x20b9, 0x20d0, 0x20d8, 0x20dd, 0x20f5, 0x2104).

Analyzed function at 0x24c0 checking matching byte pairs sequentially, terminating early on failure via multiple jumps to 0x2517.

Reconstructed the main control loop at 0x2dc0 (main) which features option parsing via strcmp, calls terminal configuration adjustments via tcgetattr/tcsetattr, and incorporates a central dispatch loop reading keyboard scan-codes using a jump table or multiple branches (0x2f28, 0x2f5f, 0x2fef, 0x3030, 0x3035, 0x304f, 0x3064, 0x3082) to perform game logic updates (e.g., 0x21e0 for shift up, 0x22e0 for rotation, 0x23c0 for slide down).

B.45 Gemini – Notepad– (Notepad--_00.json)

Confidence: 95

Reasoning

Identified the main function loop at sub_4580 based on argument verification (argc == 2 check).

Traced the control flow of sub_4580 including conditional branches at 0x459f and 0x45c0.

Mapped the infinite loop structure in sub_4580 starting from 0x45e3 which calls terminal IO and processing functions.

B.46 Gemini – Notepad– (Notepad--_O1.json)

Confidence: 70

Reasoning

Reconstructed intraprocedural CFG for multiple recovered functions based on the provided disassembly.

Identified function boundaries, basic block structures, conditional jumps, unconditional loops, and function returns.

Mapped standard control flow patterns within individual procedures without crossing function boundaries.

B.47 Gemini – Notepad– (Notepad--_O2.json)

Confidence: 90

Reasoning

Identified several prominent procedural entry points from the symbol markers including .init, sub_1350, sub_1450, sub_14e0, sub_1500, sub_1670, sub_1700, sub_1890, sub_1950, sub_19d0, sub_1a40, sub_1aa0, sub_1ae0, sub_1b40, sub_1c20, sub_1d10, sub_22d0, sub_23f0, sub_2570, sub_2650, sub_2770, sub_2850, sub_2870, sub_2880, sub_2890, sub_28b0, sub_28f0, sub_291e, sub_29f0, sub_2ab5, sub_2af0, sub_2c0e, sub_2c40, sub_2cd3, sub_2d50, sub_2e90, sub_2ef0, sub_3020, sub_3190, sub_33e0, sub_3440, sub_34f0, sub_44b0, sub_4554, sub_45d0, sub_46b0, sub_484a, sub_48c0, sub_4ba0, sub_4d10, sub_4d50, sub_4d90, main (sub_4dc0), and .fini.

Constructed intraprocedural structural edges focusing heavily on branch targets (e.g., je, jne, jmp, jl, jge) inside complex functions like sub_1350, sub_1500, sub_1700, sub_1d10, and main.

Omitted interprocedural function calls (e.g., call 10c0 snprintf@plt) and focused solely on within-boundary control flow structures.

B.48 Gemini – Notepad– (Notepad--_O3.json)

Confidence: 90

Reasoning

Analyzed the intraprocedural control flow transitions from the disassembled text section.

Isolated standard entry/exit markers, loops, conditional branches, and jumps within each distinct routine.

Omitted standard library function calls and target resolution for dynamic symbols where internal CFG structure was absent.

Identified several loops and multi-way conditional switches based on jump targets within larger functions such as sub_1d10 and sub_3510.

B.49 Gemini – Tinyhttpd (Tinyhttpd_O0.json)

Confidence: 95

Reasoning

Identified the main function and several subroutines from the disassembly section .text.

Parsed intraprocedural conditional branches (e.g., jne, je, jge, jle) and unconditional jumps (jmp) to establish basic block boundaries and connect nodes with edges.

Followed the rule that function calls do not terminate a basic block unless they are followed by control-altering instructions or act as branch targets.

B.50 Gemini – Tinyhttpd (Tinyhttpd_O1.json)

Confidence: 95

Reasoning

Reconstructed the control flow graphs for major procedures in the disassembly, including sub_1230, sub_1270, sub_12a0, sub_12e0, sub_1330 (request handler), sub_1610 (readline), sub_16f0, sub_1890, sub_1a60, sub_1ba0 (handle_cgi/request), sub_20c0, sub_21b0, sub_2220, sub_2310, sub_23e0 (startup/bind), and sub_24f0 (main loop).

Basic blocks are split on control-flow instructions such as je, jne, jmp, jb, ja, js, jle, and ret.

Function calls (like perror, exit, accept, pthread_create) do not terminate basic blocks unless accompanied by a branching instruction or terminating the function.

All edges are intraprocedural, respecting the independent function CFG requirement.

B.51 Gemini – Tinyhttpd (Tinyhttpd_O2.json)

Confidence: 95

Reasoning

Reconstructed intraprocedural CFG for multiple recovered functions based on branch and jump targets.

Isolated function call targets and model-internal branches according to specifications.

Functions mapped include: .text initialization placeholders (sub_1230), sub_1270, sub_12a0, sub_12e0, sub_1330, sub_1750, sub_1810, sub_19b0, sub_1b80, sub_1d70, sub_24a0, sub_2590, sub_2600, sub_26f0, sub_27c0, and sub_28d0 (main/server loop).

B.52 Gemini – Tinyhttpd (Tinyhttpd_O3.json)

Confidence: 95

Reasoning

Analyzed the intraprocedural control flow for all functions in the .text section.

Identified function boundaries based on entry points (endbr64, push rbp) and returns (ret).

Function sub_1230 is an initialization or error wrapper starting at 0x1230, which calls perror and exit.

Function sub_1270 handles initialization or finalization with conditional checks at 0x1281 and 0x128d.

Function sub_12a0 handles frame setup and conditionally jumps or returns.

Function sub_12e0 is __do_global_dtors_aux with checks for a flag, calling __cxa_finalize.

Function sub_1330 is a large request handling function (likely a worker or thread routine) containing numerous conditional branches for reading sockets (recv), string matching (strcasecmp), and parsing HTTP-like paths, dispatching to error/response helpers (sub_1820, sub_19c0, sub_1b90, sub_1de0).

Function sub_1760 reads a line from a socket dynamically with termination checks.

Function sub_1820 constructs and sends specific network headers and text.

Function sub_19c0 constructs and sends an alternative set of responses.

Function sub_1b90 processes a file read or socket loop, conditionally loading files via fopen and sending contents.

Function sub_1de0 executes a CGI-like fork/exec pipeline or returns error responses, tracking pipeline states.

Function sub_2530 sends basic request errors.

Function sub_2620 wraps fgets and loops until feof.

Function sub_2690 and sub_2780 send static error blocks.

Function sub_2850 sets up a server socket (socket, setsockopt, bind, listen) with multiple error handling blocks.

Function sub_2990 is the main server loop (main) which invokes sub_2850, prints status, enters an accept loop, and creates a thread via pthread_create for each incoming connection.

B.53 Gemini – bzip2 (bzip2_O0.json)

Confidence: 90

Reasoning

Identified function sub_2390 starting at 0x2390 and ending at 0x25b7.

Traced all conditional and unconditional jumps within sub_2390 to construct its basic blocks.

The disassembly becomes truncated towards the end, so focus was placed on fully recovering the complete sub_2390 function graph.

B.54 Gemini – bzip2 (bzip2_O1.json)

Confidence: 95

Reasoning

Reconstructed function-level control flow graphs for several functions extracted from the disassembly text section.

Functions identified include helper blocks around exit/perror, standard assembly initialization loops, several distinct custom logic algorithms (like a sorting routine, tree or map builder, dynamic memory layouts).

Model boundaries follow explicit conditional splits (e.g., je, jne, jg, jl, jbe, jae) and absolute jumps or sequential drops to subsequent offsets.

Intraprocedural boundaries strictly maintained; intraprocedural returns (ret) correctly mapped without outbound links to dynamic caller contexts.

B.55 Gemini – bzip2 (bzip2_O2.json)

Confidence: 90

Reasoning

Reconstructed the control flow graph for the .init section and the first function in the .text section (sub_22c0) based on conditional branches and terminal instructions.

In .init, the conditional jump at 2012 splits the flow into a branch containing a function call and a fallthrough/target block at 2016 which concludes with a return.

In sub_22c0, a comparison and conditional jump at 22ed splits execution into a block performing a write operation (22ef) and a block leading to program termination via exit (2308).

B.56 Gemini – bzip2 (bzip2_O3.json)

Confidence: 75

Reasoning

Identified internal functions corresponding to section markers and symbols within .text (e.g., .text block entry at 22c0, loops and complex block construction logic up to address 83a9).

Analyzed the basic block boundaries using branching instructions (je, jne, jg, jl, jmp, call) and sequential flow constraints.

Modeled intraprocedural structural loops and conditional jumps inside the main compression/decompression function (.text section body).

Excluded interprocedural function call destinations (e.g., fprintf@plt, fwrite@plt) from structural edges as per graph definition instructions.

B.57 Gemini – fe (fe_O0.json)

Confidence: 95

Reasoning

Reconstructed intraprocedural CFGs for multiple recovered functions based on the branch and jump targets in the disassembly.

Isolated basic blocks per function according to linear execution flow and branch targets, observing that function calls do not break a basic block.

Mapped edges based on unconditional jumps, conditional branches (je, jne, jg, etc.), switch dispatch jump tables, and returns.

B.58 Gemini – fe (fe_O1.json)

Confidence: 95

Reasoning

Reconstructed intraprocedural CFG for multiple functions in the binary by identifying split points at conditional/unconditional branches, returns, and jumps.

Functions like .init, putchar@plt, and several internal subroutines (sub_1120, sub_1180, sub_11b0, sub_11f0, sub_1250, sub_1320, sub_1360, sub_1430, etc.) were parsed into sequential basic blocks.

Ensured all function calls do not terminate their respective basic blocks but remain part of them, keeping control flow strictly intraprocedural.

B.59 Gemini – fe (fe_O2.json)

Confidence: 95

Reasoning

Analyzed the intraprocedural control flow transitions from the disassembly of multiple functions.

Functions like sub_1000 (.init), sub_1120, sub_1180, sub_11b0, sub_11f0, sub_1230, sub_1240, sub_1250, sub_1320, sub_1360, sub_13b0, sub_13d0, sub_13e0, sub_13fe, sub_1420, sub_1430, sub_14b0, sub_1540, sub_15e0, sub_1670, sub_1750, sub_1820, sub_1a50, sub_1ad0, sub_1b50, sub_1c20, sub_1c90, sub_1cf0, sub_1d60, sub_21a0, sub_21c0, sub_21d0, sub_21f0, sub_22c0, sub_22f0, sub_29a0, sub_29d0, sub_29f0, sub_3730, sub_38e0, sub_39c0 (main), sub_3b30, sub_3c00, sub_3d10, sub_3da5, sub_4058 (.fini) have been mapped.

Basic blocks are split by conditional branches (je, jne, jb, ja, jl, jge, etc.), unconditional jumps, and return instructions.

Interprocedural function calls are treated as normal instructions inside a block and do not introduce edges.

B.60 Gemini – fe (fe_O3.json)

Confidence: 95

Reasoning

Analyzed the intraprocedural control flow transitions across all recovered functions within the .text section.

Correctly identified function boundaries, call sites, and branch terminations including conditional jumps (je, jne, jl, jge, jb, jne), unconditional jumps (jmp), and function returns (ret).

Isolated basic blocks per function sequentially without drawing interprocedural edges for functions like sub_1120, sub_1180, sub_11b0, sub_11f0, sub_1250, sub_1320, sub_1360, sub_13e0, sub_1430, sub_14d0, and others.

B.61 Gemini – kilo (kilo_O0.json)

Confidence: 95

Reasoning

Reconstructed intraprocedural control flow for all identifiable functions in the .text section.

Functions extracted based on entry points (such as push rbp, mov rbp, rsp or endbr64) and separated by blocks of nop/ret.

Identified branch destinations from relative jump instructions (e.g., je, jne, jmp) to compute target blocks accurately.

Modelled switch dispatch tables and conditional bounds logic accurately within the specific processing functions (e.g., 2fa0, 3010).

Excluded function call edges and external/plt entries to ensure strict intraprocedural constraints.

B.62 Gemini – kilo (kilo_O1.json)

Confidence: 100

Reasoning

Analyzed the function at 0x23c0 (tcgetattr/tcsetattr handling) for linear flow and standard block termination.

Analyzed the function at 0x2490 (read loop and complex multi-way conditional handling for escape sequence decoding) to split basic blocks on conditional jumps (je, jne, ja, jae) and the indirect jump (jmp rax) resulting from a switch dispatch.

Analyzed the function at 0x2630 (vsnprintf / sscanf helper) mapping out conditional branches around read loops and validation checks.

Analyzed the function at 0x26f0 (ioctl and dual-sscanf/snprintf flow) checking branches on ioctl result, read success, and string conversions.

Analyzed the function at 0x28e0 (ctype_b_loc / memchr verification) separating logic based on test/je/jne instructions.

Analyzed the function at 0x2930 (simple structural check with early returns) identifying basic blocks separated by conditional jumps.

Analyzed the main entry point function at 0x4d10 (argument verification, signal setup, tracking loop) isolating blocks up to the infinite interactive execution loop.

B.63 Gemini – kilo (kilo_O2.json)

Confidence: 95

Reasoning

Analyzed the provided disassembly to identify individual function entry points and boundaries.

Mapped out the basic blocks within each function based on control-flow instructions such as jmp, je, jne, ja, jae, jl, jle, and ret.

Identified function calls (e.g., call 2110 ¡read@plt¿) and treated them as part of the continuous basic block, adhering to the requirement that call instructions do not terminate basic blocks.

Isolated intraprocedural control flow within functions like sub_2000, sub_2270, sub_23c0, sub_2490, sub_2630, sub_26e7, and main (sub_51b0).

Ensured no interprocedural edges were created between distinct functions.

B.64 Gemini – kilo (`kilo_O3.json`)

Confidence: 95

Reasoning

Analyzed the provided disassembly to identify individual function entry points and basic block boundaries based on control flow instructions (jumps, branches, returns, and calls).

Extracted internal intraprocedural structures for key functions, ignoring interprocedural transitions (calls/returns between different functions).

Successfully mapped basic blocks sequentially, resolving conditional jumps (e.g., je, jne, jg, jl) and unconditional jumps into explicit directional edges.

Functions recovered include .init, free@plt, and various numbered sub-routines in the .text section matching pattern rules.

B.65 Gemini – lsh (`lsh_O0.json`)

Confidence: 95

Reasoning

Analyzed the provided x86_64 disassembly to extract function boundaries and build intraprocedural CFGs.

Identified function starting at 0x1210 (possibly 'builtin_cd'). It has conditional branches at 0x1225 and 0x1251, leading to basic blocks terminating at 0x126b (ret).

Identified function starting at 0x1270 (possibly 'builtin_help'). Contains a loop back edge from 0x12e7 to 0x12ad based on the comparison and conditional branch at 0x12bf.

Identified short helper functions at 0x1310 and 0x1320.

Identified function starting at 0x1330 (possibly 'execute_command_fork'). Contains a loop at 0x138b based on the conditional jump at 0x13ca.

Identified function starting at 0x13e0 (possibly 'execute_builtin_or_external'). Has a loop from 0x145f back to 0x1406.

Identified function starting at 0x1480 (possibly 'read_line'). Loop from 0x156d to 0x14d2.

Identified function starting at 0x1580 (possibly 'split_line'). Loop from 0x1682 back to 0x15e8.

Identified function starting at 0x16b0 (possibly 'sh_loop'). Loop from 0x16fe back to 0x16b8.

Identified function starting at 0x1710 ('main'). Sequential flow calling sh_loop and returning.

B.66 Gemini – lsh (`lsh_O1.json`)

Confidence: 95

Reasoning

Identified sub_1220 (likely a cd command implementation), sub_1270 (help command), sub_12e0, sub_12f0, sub_1300 (process spawning / fork / execvp loop), sub_1390 (command dispatcher), sub_1410 (line reader), sub_14c0 (tokenizer), and main (starting at 15a0 or 1660, with 15a0 being the main shell loop).

Analyzed control flow transitions based on conditional branches, unconditional jumps, and function boundary instructions (returns/calls).

Constructed intraprocedural edges for all identified functions.

B.67 Gemini – lsh (`lsh_O2.json`)

Confidence: 95

Reasoning

Analyzed the intraprocedural control flow from the provided disassembly.

Identified several functions, including helper functions for a shell-like application (e.g., handling chdir, help, command execution, input reading, tokenization, and main loop).

Function at 0x1220 handles 'cd' command: branches at 1228 (je 123a), 1231 (jne 125c), with 125a and 1268 jumping back to 1233.

Function at 0x1270 prints help or prompt info and returns sequentially.

Function at 0x12f0 and 0x1300 are simple stub/return functions.

Function at 0x1310 handles process forking and execution: branches at 131e (je 1372), 1320 (js 1364), loop at 1355 (jl 1330), and branches inside child path at 1380.

Function at 0x13a0 matches commands against builtins and executes or dispatches via an indirect jump table at 141e.

Function at 0x1430 reads input line-by-line (getc loop) with dynamic resizing via realloc: loop branches at 146c (je 14c3), 1471 (je 14d9), and 147d (jg 1460), 1497 (jne 1460).

Function at 0x14e0 tokenizes the input string: dynamic token loop with branches at 150e, 1543, 1552, 1567.

Function at 0x15c0 is the main shell loop: loops back from 1685 (jne 15f3).

Function at 0x16a0 is the main entry wrapper calling the main shell loop.

B.68 Gemini – lsh (`lsh_O3.json`)

Confidence: 95

Reasoning

Analyzed the provided assembly code and parsed individual functions based on entry points, branch targets, and return statements.

Identified several intraprocedural functions including .init (1000), .text (1130), sub_1160 (1160), sub_1190 (1190), sub_11d0 (11d0), sub_1210 (1210), sub_1220 (1220), sub_1280 (1280), sub_1300 (1300), sub_1310 (1310), sub_1320 (1320), sub_13b0 (13b0), sub_1440 (1440), sub_14f0 (14f0), sub_15d0 (15d0), main (16b0), and .fini (16bc).

Divided each function into basic blocks at control flow alters like je, jne, jmp, and ret instructions, while ignoring function call destinations to keep the CFG purely intraprocedural.

B.69 Gemini – md5-c (`md5_O0.json`)

Confidence: 95

Reasoning

Analyzed the provided disassembly to identify individual functions and their internal basic block control flow structures.

Isolated functions: .init, sub_1020, free, fread, strlen, printf, malloc, __cxa_finalize, sub_1090, sub_10c0, sub_1130, sub_1170, sub_1180, sub_11e0 (main-like), sub_1260, sub_1290, sub_12e0, sub_1400, sub_1590, sub_1770, sub_17e0, and .fini.

Created intraprocedural edges based on branch instructions (je, jne, jmp, jae, jle, jge, jbe), making sure not to cross function boundaries.

B.70 Gemini – md5-c (`md5_O1.json`)

Confidence: 95

Reasoning

Reconstructed intraprocedural CFGs for the individual functions found in the .text section: sub_10a0, sub_10d0, sub_1100, sub_1140, sub_1180, sub_1190, main_or_similar (11e0), rotate_left (12b0), md5_init (12c0), md5_update (12e0), md5_transform (1370), md5_final (14d0), md5_string (15c0), and md5_file (17c0).

Modelled loop back-edges accurately based on conditional jumps to earlier targets (e.g., in md5_transform 1421 -¿ 1494 or jump back to 13d0/1423).

Calls to PLT or internal functions are contained within their respective basic blocks without introducing interprocedural or call edges.

B.71 Gemini – md5-c (`md5_O2.json`)

Confidence: 90

Reasoning

Identified several functions from the disassembly including sub_1190, sub_1290, sub_1300, sub_1310, and sub_1330.

Reconstructed the control flow for sub_1290 based on conditional jump (jl 12ce) and loop (jne 12b0).

Reconstructed the control flow for sub_1330 involving multiple nested loops and conditional jumps.

B.72 Gemini – md5-c (`md5_O3.json`)

Confidence: 95

Reasoning

Reconstructed intraprocedural CFG for multiple functions in the disassembly.

Functions like sub_1190 contain sequential basic blocks with a single exit point (putchar@plt via jmp).

Functions like sub_1290 contain conditional jumps (jl 12ce, jne 12b0), forming loops and branches.

Functions like sub_1330 contain conditional branches (je 142e, jbe 1420, jne 1370) that form complex loops and conditional blocks.

Functions like sub_1430 (MD5/cryptographic step-like layout) contain conditional jumps (je 1554, je 1520, je 1490, jne 1540).

All function calls are treated as internal instructions and do not break basic blocks.

Interprocedural edges (calls/returns) are omitted per rules.

B.73 Gemini – pdpmake (`pdpmake_O0.json`)

Confidence: 95

Reasoning

Analyzed the intraprocedural control flow within the given x86_64 disassembly routines.

Functions identified include the .init section, various PLT stubs, error initialization, formatting, custom path/string management routines, and core logic components.

Basic blocks are delineated based on execution-altering instructions such as conditional jumps (je, jne, jge, jl, jb, ja, jae), unconditional jumps (jmp), returns (ret), and trap instructions (hlt).

Linear sequences of instructions (including function call instructions like call) are preserved inside their respective basic blocks.

No interprocedural edges (edges bridging distinct functions) have been included, strictly respecting the intraprocedural bounds.

B.74 Gemini – pdpmake (`pdpmake_O1.json`)

Confidence: 75

Reasoning

The input is a single binary module containing multiple functions, including .init, various PLT stubs, and the primary function 'error@@Base'.

Each function has been analyzed independently to reconstruct its control flow.

Basic blocks are identified by identifying entry points (targets of branches, calls, and sequential flow) and terminating instructions (branches, jumps, calls, returns).

Function calls, such as those to PLT stubs (e.g., exit@plt, printf@plt), are modeled as part of the basic block containing the call instruction, as they do not terminate the block.

Conditional and unconditional jumps establish edges between basic blocks.

Edges are strictly intraprocedural, ignoring cross-function transitions or call/return relationships.

Some disassembly includes 'data16' or NOP paddings which are ignored during CFG construction.

Confidence is 75 due to complex indirect jumps (like 'jmp rax' in some sections) where potential targets might not be fully resolvable without dynamic analysis, necessitating a conservative approach to edge creation.

B.75 Gemini – pdpmake (`pdpmake_O2.json`)

Confidence: 75

Reasoning

The input contains a complex ELF binary disassembly, including .init, .plt, and .text sections.

Functions like error@@Base contain multiple basic blocks with jumps, conditional branches, and function calls.

The construction identifies multiple entry points (e.g., error@@Base at 23b0, and other sub-routines).

Basic blocks are separated by control flow instructions such as jne, je, call, ret, and unconditional jumps.

The CFG reconstruction follows intraprocedural rules, ensuring edges do not cross function boundaries.

Some indirect jumps (e.g., jmp rax at 260f) create ambiguity in edge destination, but the structural flow of surrounding blocks is captured.

B.76 Gemini – pdpmake (pdpmake_O3.json)

Confidence: 75

Reasoning

Reconstructed basic blocks by identifying sequential instructions that terminate at control-flow instructions (jmp, je, jne, ret, call) or branch targets.

The function error@@Base (starting at 23b0) contains several distinct logic paths based on jump and call instructions, leading to multiple basic blocks (B0-B_final).

Many basic blocks end in jmp or ret, forming clear entry/exit points for the CFG.

Some edges were inferred based on conditional branch targets (e.g., je, jne) and unconditional jumps.

Internal function calls were treated as part of the current basic block as per instructions, and edges represent control flow within error@@Base.

B.77 Gemini – wak (wak_O0.json)

Confidence: 75

Reasoning

The function at 2390 is the program entry point (main).

Basic blocks are defined by branch targets and control-flow instructions.

Function calls are treated as internal to basic blocks per instructions.

The CFG was reconstructed by identifying jump/branch targets (e.g., 23d1, 23dd) and return points within .text.

Some targets are indirect jumps (e.g., jmp rax), which limits absolute certainty in edge mapping.

B.78 Gemini – wak (wak_O1.json)

Confidence: 75

Reasoning

The input contains multiple distinct functions and code sections (.init, .plt, .plt.got, .text).

Each function is identified by a unique address and contains its own sequence of basic blocks.

Basic blocks are delineated by control-flow instructions (jmp, je, jne, call, ret) as per the rules.

Function call instructions are included within basic blocks, not as separate nodes or edges, maintaining intraprocedural graph constraints.

The graph is constructed by analyzing unconditional and conditional jump targets for edges within the same function.

Some uncertainty exists regarding the exact boundaries of very large functions or complex jump tables, so confidence is set to 75%.

B.79 Gemini – wak (wak_O2.json)

Confidence: 75

Reasoning

The input contains multiple functions (including .init, plt entries, and numerous .text functions).

Basic blocks are identified by control-flow altering instructions (call, ret, jmp, je, jne, jge, ja, jb).

Function calls are treated as part of the basic block, not as edges, as per instructions.

Edges represent intraprocedural transfers (conditional and unconditional branches).

The reconstruction is partial, focusing on the main logic structures in .text.

Functions lacking clear boundaries (e.g., jump tables, complex switch-like logic) were analyzed based on visible control flow.

Some uncertain branches (e.g., computed jumps) were omitted to maintain soundness.

B.80 Gemini – wak (`wak_O3.json`)

Confidence: 75

Reasoning

The input is a complete x86-64 assembly listing including .plt, .text, and other sections.

Basic blocks were identified by locating entry points (like function start addresses) and control-flow instructions (jumps, calls, returns).

Calls to library functions (e.g., fwrite@plt, vfprintf@plt) are treated as part of the calling basic block, and no interprocedural call/return edges were modeled.

Multiple functions were identified (e.g., .init, the main code starting at 2390, and various sub-routines).

Edges represent intraprocedural transfers; branches and jumps are resolved to their target addresses within the same function.

Uncertain branches (e.g., indirect jumps like 'jmp rax') result in the omission of only the uncertain edge, adhering to the provided instructions.

B.81 Grok – 2048.c (`2048_O0.json`)

Confidence: 65

Reasoning

Identified multiple functions via standard x86-64 prologues (push rbp; mov rbp,rsp) and ret instructions in .text section.

Main function recovered at 0x21e0 with extensive control flow including argument parsing, game loop with input handling, conditional branches for moves (w/a/s/d), and win/lose checks.

Other functions identified at addresses like 0x1200 (byte manipulation), 0x1360 (digit count), 0x1390 (print), 0x17e0 (matrix rotate), 0x1c90 (random fill), 0x1ea0 (win check), etc., based on calls and structure.

Basic blocks split at branches (jumps, conditional j* , ret), calls do not split BBs per rules. Intra-function edges only.

Partial reconstruction due to large number of small functions and complex loops; omitted uncertain indirect edges.

B.82 Grok – 2048.c (`2048_O1.json`)

Confidence: 45

Reasoning

Disassembly lacks explicit function symbols in .text; identified main at 0x29c0 based on argc/argv handling and high-level structure.

Multiple intraprocedural basic blocks recovered from branch targets (je, jne, jmp, etc.) and fallthroughs within identifiable subroutines starting at addresses like 0x12e0, 0x16f0, 0x1730, etc.

Partial CFG due to dense code with many small helper functions; focused on main and key helpers with clear control flow; omitted uncertain inter-block edges where targets ambiguous.

No interprocedural edges created; calls to PLT ignored for CFG purposes.

B.83 Grok – 2048.c (`2048_O2.json`)

Confidence: 65

Reasoning

Disassembly provided covers .text with multiple identifiable functions via prologues (push rbp etc.), ret instructions, and call targets.

Main function identified at 0x2bf0 (sub_2bf0), large game loop with keyboard input handling (getc), grid operations, and terminal mode changes.

Helper functions extracted: sub_1200 (grid printing), sub_12e0 (grid update/print loop), sub_2580 (random fill), sub_28c0 (validation/check), sub_2820 (terminal attrs), smaller grid manipulators like sub_1c60, sub_1d50 etc.

Basic blocks split at branches (je/jne/jmp), returns, and targets; calls do not split blocks per rules. Intra-function edges only.

Some indirect/conditional edges approximated; full coverage of major paths in main and helpers; minor uncertainty in dense loops.

B.84 Grok – 2048.c (2048_O3.json)

Confidence: 65

Reasoning

Disassembly contains multiple distinct functions identifiable by standard prologues (endbr64/push rbp), ret instructions, and call targets.

Basic blocks constructed by splitting at conditional jumps (je/jne/ja), unconditional jmp, ret, and branch targets. Consecutive instructions without control flow form single blocks.

Functions identified: sub_1110 (entry), sub_1200, sub_12b0, sub_12e0 (large logic), sub_1f90, sub_1fe0, sub_2130, sub_2180, sub_21e0, sub_22e0, sub_23c0, sub_24c0, sub_2520, sub_2570, sub_27f0, sub_2a70, sub_2a90, sub_2b30, sub_2d90, sub_2dc0 (main).

Intra-function edges from jumps/conditionals only; no interprocedural or call/return edges per rules. Partial coverage for dense sections like sub_12e0 due to length/complexity.

No empty CFG as substantial recoverable control flow exists.

B.85 Grok – Notepad– (Notepad--_O0.json)

Confidence: 45

Reasoning

The disassembly contains startup code (.init, .text entry at 0x1230), PLT stubs, and multiple functions identifiable by standard prologues (push rbp; mov rbp,rsp) and epilogues (pop rbp; ret).

Basic blocks extracted from visible control flow: conditional jumps (je/jne), unconditional jumps (jmp), loops, and returns. Many functions like buffer append (around 1320), terminal setup (1460,1480), file loading (1780), and input handling are present.

Full reconstruction limited by input truncation and scale; partial intraprocedural CFGs for key functions (e.g., sub_1320, sub_1530, sub_1780) with identified branches. No interprocedural edges included per rules.

B.86 Grok – Notepad– (Notepad--_O1.json)

Confidence: 45

Reasoning

The disassembly is a large .text section with multiple intraprocedural functions identifiable by standard prologues (push rbp, etc.) and epilogues (pop, ret).

Basic blocks derived from conditional jumps (je, jne, jl, etc.), unconditional jumps, and ret instructions. Function boundaries inferred from call targets and entry points.

Many functions like buffer management, file I/O, terminal control (tcgetattr/tcsetattr), and editing logic present. Partial CFG covers visible control flow; full reconstruction limited by truncation and label reuse.

No interprocedural edges included; only intra-function branches and fall-throughs.

B.87 Grok – Notepad– (Notepad--_O2.json)

Confidence: 40

Reasoning

Disassembly provided with no explicit function symbols in .text beyond PLT and init/fini; identified candidate function starts via common prologues (push rbp) and internal call targets.

Reconstructed intraprocedural CFGs for several identifiable functions based on branch/jump targets, conditional branches, and returns within address ranges.

Basic blocks split at control flow instructions (jumps, branches, calls to external but kept intra for blocks, rets); many edges recovered from je/jne/jmp etc., but full coverage limited by size and truncation.

No interprocedural edges; treated each candidate function independently.

B.88 Grok – Notepad– (Notepad--_O3.json)

Confidence: 45

Reasoning

Disassembly shows multiple functions in .text with standard x86-64 prologues (push rbp, etc.) and control flow via conditional jumps (je, jne, jg, jl), unconditional jmp, and ret.

Basic blocks identified by branch targets and fall-throughs within visible functions like sub_1350 (string append/buffer), sub_1500 (init), sub_1700 (file load with getline loop), sub_1890 (cleanup).

Intraprocedural edges from jumps/branches within each function; calls to PLT ignored per rules; partial reconstruction due to truncation and label offsets.

No interprocedural edges; focused on confidently recoverable blocks and flows from provided disassembly.

B.89 Grok – Tinyhttpd (Tinyhttpd_O0.json)

Confidence: 65

Reasoning

Identified multiple functions in .text section via address patterns and stack frames (push rbp, sub rsp).

Parsed basic blocks by locating branch targets (labels like je, jmp destinations), conditional jumps, and returns.

Reconstructed intraprocedural edges for key functions like the server loop at 2920, request handler at 1390, and utilities; ignored PLT stubs and interprocedural calls.

Basic blocks split only at control flow instructions or targets; calls do not split blocks per rules.

Partial coverage due to large disassembly size and some complex loops/conditions, but core flow recovered.

B.90 Grok – Tinyhttpd (Tinyhttpd_O1.json)

Confidence: 65

Reasoning

Identified distinct functions via prologues, stack adjustments, and call targets in .text section: error handler at 1230, client_handler at 1330, read_line at 1610, send_404 at 16f0, send_500 at 1890, serve_file at 1a60, handle_request at 1ba0, send_error at 20c0, send_file at 21b0, send_alt_error at 2220, send_header at 2310, create_socket at 23e0, main at 24f0.

Basic blocks determined by branch targets (je, jne, jl, ja, jmp, etc.), loop entries, and return points; calls do not split blocks per rules; intra-function jumps and conditional branches define edges.

Partial reconstruction prioritizing high-confidence branches; omitted uncertain indirect edges; all functions treated independently with no interprocedural edges.

B.91 Grok – Tinyhttpd (Tinyhttpd_O2.json)

Confidence: 65

Reasoning

Disassembly shows multiple functions identifiable by standard x86-64 prologues (push rbp, sub rsp), epilogues (pop rbp, ret), and call sites in .text section starting at 0x1230.

Major functions recovered: handle_client at 0x1330 (large request parser with recv loops), process_request at 0x1d70, main at 0x28d0, setup_socket at 0x27c0, and several error/send helpers (0x1810, 0x19b0, etc.).

Basic blocks identified by branch targets, conditional jumps (je, jne, jle, etc.), unconditional jmp, ret instructions; calls do not split blocks. Intraprocedural edges only; no inter-function edges.

Partial reconstruction due to complex recv loops and conditional paths in HTTP handling; some fallthroughs and loop back-edges included where clear from offsets.

B.92 Grok – Tinyhttpd (Tinyhttpd_O3.json)

Confidence: 65

Reasoning

Disassembly shows multiple functions in .text section identified by prologues (push rbp etc.), rets, and call targets.

Functions identified: sub_1230 (error), sub_1330 (client_handler), sub_1760 (recv_line), sub_1820 (send_error), sub_19c0 (send_forbidden), sub_1b90 (serve_file), sub_1de0 (handle_request), sub_2530 (bad_request), sub_2620 (send_content), sub_2690 (send_internal_error), sub_2780 (send_http_headers), sub_2850 (create_server_socket), sub_2990 (main).

Basic blocks derived from branch targets (je, jne, jmp, jle etc.), fall-throughs, and ret instructions within each function. No interprocedural edges. Partial reconstruction due to large size and label offsets.

Edges follow conditional/unconditional jumps and sequential flow within functions.

B.93 Grok – bzip2 (bzip2_O0.json)

Confidence: 40

Reasoning

The disassembly is from a stripped ELF binary (likely bzip2 or similar compressor) with .text section containing multiple functions identified by standard x86-64 prologues (push rbp; mov rbp,rsp).

PLT stubs and init/fini sections have simple linear CFGs with conditional jumps and returns. Main .text functions show complex control flow with loops, conditionals, and calls to library functions.

Due to the large truncated disassembly (800k+ bytes) and lack of symbol names, only partial intraprocedural CFG reconstruction is feasible. Identified entry points at 0x22a0, 0x2390, 0x25c0 etc. Basic blocks split at branches (je, jmp, jge, ret). No interprocedural edges included.

CFG represents key structures from visible code: init routines, compression/decompression helpers with loops over buffers and bit operations.

B.94 Grok – bzip2 (bzip2_O1.json)

Confidence: 45

Reasoning

Disassembly provided is extensive but lacks standard function symbols (stripped binary with relative PLT labels). Identified multiple function prologues via push rbp/sub rsp patterns at addresses like 0x2620, 0x3840, 0x41f0 etc.

Basic blocks derived from control flow instructions (je/jne/jmp/ret/conditional branches) within .text section starting at 0x22c0. Intra-procedural edges follow branch targets within same function.

Limited to key functions due to file size/truncation; no interprocedural edges per rules. Partial CFG focuses on identifiable entry points and visible branches.

Confidence moderate due to massive disassembly volume and label artifacts preventing full automated parsing.

B.95 Grok – bzip2 (bzip2_O2.json)

Confidence: 40

Reasoning

The disassembly is from a stripped binary with limited function symbols; most code is under .text starting at 0x22c0 containing multiple inlined or unlabeled functions.

Identified basic blocks from visible control flow instructions (branches, jumps, returns) in the provided .text section excerpts.

Reconstructed intraprocedural CFGs for prominent entry points like the one at 0x22c0 (likely main or entry), 0x2320, 0x23a0 etc., focusing on conditional jumps (je/jne) and unconditional jumps.

PLT stubs and .init/.fini have simple CFGs but are library-related; prioritized user code in .text.

Edges follow standard rules: terminate BBs at branches/returns; no interprocedural edges.

B.96 Grok – bzip2 (bzip2_O3.json)

Confidence: 45

Reasoning

Disassembly provided contains .text section with multiple unlabeled subroutines starting at addresses like 0x22c0, 0x2320, 0x23a0, 0x2530 (_start), 0x2620 (large function likely main processing), and ending at .fini.

Basic blocks identified by linear sequences terminated by branches (je, jne, jmp, call does not terminate), returns, etc. Intra-procedural edges only within each subroutine.

Functions inferred by entry points with standard prologues (push rbp etc.) and ret instructions. PLT entries treated as separate trivial functions but CFG focuses on .text. Partial reconstruction due to truncation and lack of symbols.

Nodes numbered per subroutine (e.g., sub_22c0::B0). Edges from conditional/unconditional branches within same function.

B.97 Grok – fe (fe_O0.json)

Confidence: 60

Reasoning

The disassembly contains standard CRT startup code at 0x1120 (_start) with conditional branches and calls.

Multiple functions identified by push rbp / mov rbp,rsp prologues and ret instructions, such as at 0x1210, 0x1220, 0x12f0, etc.

Intraprocedural edges extracted from je/jne/jmp/call/ret within each identified function boundary; PLT stubs treated as separate trivial functions.

CFG is partial due to large size and some unresolved indirect jumps, but covers key basic blocks and control transfers.

B.98 Grok – fe (fe_O1.json)

Confidence: 60

Reasoning

The disassembly shows multiple intraprocedural control flows with conditional jumps (je, jne, jb, etc.), loops, and returns in .text section.

Identified basic blocks around key addresses like 0x1250 (multi-register push function with branches), 0x1320, 0x1360, 0x1430, etc., based on prologues and ret instructions.

Edges constructed from visible branches (e.g., je to targets) within apparent function boundaries; calls ignored as per rules; partial reconstruction due to large/truncated input but sound for visible parts.

B.99 Grok – fe (fe_O2.json)

Confidence: 45

Reasoning

Disassembly lacks symbol names; functions identified by prologues (push rbp etc.), epilogues, and address ranges where control flow stays local.

Major functions reconstructed: init-like at 1120/1150, error_handler at 1250, parser/eval at ~1d60/2a00/39c0, with basic blocks split at branches/jumps/returns.

Intra-procedural edges from conditional jumps (je/jne), unconditional (jmp), and fall-throughs within identified function bounds. Calls ignored per rules.

Partial CFG due to size/truncation; focused on key .text routines with clear CF.

B.100 Grok – fe (fe_O3.json)

Confidence: 40

Reasoning

Disassembly shows .text section starting at 0x1120 with mixed code, PLT stubs, and no clear function symbols beyond sections and PLT entries.

Identified potential entry points and branches (je, jmp, call, ret) within .text for basic block construction.

Reconstructed minimal intraprocedural CFG for main .text block and key subroutines based on visible control transfers; many edges inferred from conditional jumps and sequential flow.

Limited by unlabeled functions and truncation; focused on intraprocedural edges only, avoiding calls/returns.

B.101 Grok – kilo (`kilo_00.json`)

Confidence: 45

Reasoning

Disassembly provided as .asm dump with .text section containing multiple functions identifiable by push rbp/mov rbp,rsp prologues and ret instructions.

Identified key functions: entry/_start at 0x2270, terminal setup ~0x23b0, get_key ~0x2490, render ~0x4100, main loop ~0x4880.

Basic blocks split at branches (je, jne, jmp, conditional), returns; calls do not split blocks. Partial reconstruction due to truncation and label offsets; focused on prominent intraprocedural flows.

No interprocedural edges; only within-function control transfers.

B.102 Grok – kilo (`kilo_01.json`)

Confidence: 40

Reasoning

The disassembly is from a stripped binary with no user function symbols in .text, only PLT entries and init/fini.

Identified entry points and common function patterns (e.g., around 0x2270 for _start-like, 0x4d10 for main-like, several subs with push rbp).

Basic blocks derived from branch targets (je, jmp, call targets within function), ret instructions, and fall-throughs. Full CFG reconstruction limited by size and lack of symbols, so partial with representative functions.

Intraprocedural only: no call/return edges across functions. Used sub_ identifiers for unnamed functions.

B.103 Grok – kilo (`kilo_02.json`)

Confidence: 65

Reasoning

Disassembly reveals multiple distinct functions in .text section identifiable by standard prologues (push rbp, endbr64, register saves) and epilogues (ret, pop sequences).

Basic blocks constructed by identifying linear sequences terminated by branches (je, jne, jmp, conditional jumps), returns, and targets of jumps (labels like 24b0, 25d7).

Intraprocedural edges only: calls treated as non-terminating within BBs; no inter-function edges or call/return modeling.

Key functions recovered: entry at 0x2270, terminal setup at 0x23c0, key reading at 0x2490 and 0x2630, screen/ANSI handling at 0x26f0, buffer processing at 0x2970, main loop logic around 0x4020 and 0x4950.

Truncation in provided disassembly limits full recovery of later functions, but core control flow (conditionals, loops over input, state updates) is reconstructible with high local accuracy.

B.104 Grok – kilo (`kilo_03.json`)

Confidence: 35

Reasoning

Disassembly is stripped with no symbol table; functions identified by standard prologues (push rbp/sub rsp), call sites, and return patterns.

Basic blocks split at branches (je/jne/jmp/conditional), returns, and targets; calls do not split blocks per rules.

Partial reconstruction due to truncation in input and complexity; focused on .text entry and key visible functions like input handler at 24a0 and terminal setup at 23c0; intra-function edges only.

Many edges recovered from explicit jumps; indirect/switch cases (e.g., at 25e2) partially resolved where targets visible.

B.105 Grok – lsh (`lsh_O0.json`)

Confidence: 75

Reasoning

Disassembly covers complete .text section with clear function boundaries via standard prologues (push rbp; mov rbp,rsp) and ret instructions.

Basic blocks identified per rules: leaders at function entries, branch targets, and post-branch instructions; maximal linear sequences without internal branches.

Intraprocedural only: no call/return edges modeled; functions like main, shell_loop, read_line, etc. reconstructed independently based on control transfers (jne, je, jmp).

All major functions (change_directory, print_menu, execute_command, command_dispatch, read_line, tokenize, shell_loop, main) analyzed; init stubs included where relevant.

B.106 Grok – lsh (`lsh_O1.json`)

Confidence: 65

Reasoning

Identified multiple functions in .text section based on entry points, ret instructions, and call targets: sub_1130 (_start), sub_1160, sub_11d0, sub_1220 (cd), sub_1270 (help), sub_12e0, sub_12f0, sub_1300 (run_external), sub_1390 (dispatch), sub_1410 (read_line), sub_14c0 (tokenize), sub_15a0 (shell_loop), sub_1660 (main).

For each function, identified leaders as function entries, branch targets, and fall-throughs after terminating instructions (jumps, conditional branches, rets). Grouped into maximal basic blocks without internal control flow.

Constructed intraprocedural edges only; calls do not create edges or split blocks; no inter-function edges. Partial recovery for complex loops and error paths where targets clear.

Small stub functions have single-block CFGs. Complex functions like shell_loop and read_line have loops with multiple BBs and back-edges.

B.107 Grok – lsh (`lsh_O2.json`)

Confidence: 65

Reasoning

Disassembly provided is complete for .text section with clear function boundaries identifiable via prologues, calls, rets, and padding nops.

Basic blocks constructed per rules: maximal sequences ending at branches/returns/jumps; calls do not terminate BBs; leaders include entries and branch targets.

Intraprocedural only: no call/return edges between functions; multiple small utility functions and init code included where CF identifiable.

Some small functions have single-BB CF; main loop and dispatch have multiple due to conditionals and jump table.

B.108 Grok – lsh (`lsh_O3.json`)

Confidence: 65

Reasoning

Disassembly provided allows clear identification of multiple functions via entry points, prologues (push rbp etc.), epilogues (pop/ret), and call targets.

Basic blocks identified by linear sequences ending at branches (je, jne, jl, jmp), calls (not terminating BB), or returns. Targets of jumps define new BB starts.

Intraprocedural only: no call/return edges modeled. Functions: _start, init variants, cd, help, run_cmd, handle_cmd, read_line, tokenize, shell_loop, main.

Some padding/nops treated as part of prior BB. All recoverable intra-function flows included; minor indirects via tables noted but edges from known jumps added.

B.109 Grok – md5-c (`md5_O0.json`)

Confidence: 75

Reasoning

Disassembly provided is complete for .text section with clear function boundaries identifiable via standard prologues (push rbp; mov rbp,rsp), epilogues (pop rbp; ret), and cross-references via calls/jumps.

Functions identified: _start (1090), __libc_csu_init variants (10c0,10f0,1130), print_bytes (1180), main (11e0), rotate_left (1260), md5_init (1290), md5_update (12e0), md5_transform (1400), md5_final (1590), md5 (1770), md5_file (17e0).

Basic blocks constructed per intraprocedural rules: maximal straight-line sequences ending at branches/returns/jumps. Loops and conditionals create multiple blocks per function. No interprocedural edges or call/return modeling.

All major loops (e.g., in md5_update, md5_transform, main argument processing) and conditionals fully recovered with high fidelity.

B.110 Grok – md5-c (`md5_O1.json`)

Confidence: 65

Reasoning

Disassembly provided covers standard ELF startup, MD5 implementation functions, and utility routines with clear prologues and control flow instructions.

Functions identified by prologues (endbr64/push rbp), call targets, and ret instructions; basic blocks split at branches (jcc, jmp, ret) and targets.

Intra-function edges from conditional jumps, unconditional jumps, fall-throughs; no inter-procedural edges or calls included per rules.

Main, MD5 update/compress/final, file/string handlers reconstructed with high fidelity; library stubs and small inits have simple CFGs.

B.111 Grok – md5-c (`md5_O2.json`)

Confidence: 65

Reasoning

Identified key functions from prologues, epilogues, and call targets in .text: main at 0x1290, print_hex at 0x1190, md5_init at 0x1310, md5_update at 0x1330, md5_transform at 0x13c0, md5_final at 0x1520, md5_string at 0x1610, md5_file at 0x1830, and startup at 0x10a0.

Basic blocks split at branches (je, jne, jmp, jbe, etc.), returns, and targets. Loops in MD5 processing create multiple BBs per function.

Intraprocedural only: no call/return edges modeled. Partial reconstruction due to dense code and offsets; focused on clear control flow in main and core MD5 funcs.

Rol at 0x1300 is trivial single BB. Many small init/fini stubs.

B.112 Grok – md5-c (`md5_O3.json`)

Confidence: 70

Reasoning

Disassembly parsed to identify distinct functions via prologues, call targets, and entry points in .text section.

Basic blocks constructed by identifying linear sequences terminated by branches (jumps, conditional, ret) or targets.

Intraprocedural only: no call/return edges modeled; functions treated independently.

Main, MD5-related functions (update, compress, final, etc.) recovered with their internal CFGs.

B.113 Grok – pdpmake (pdpmake_00.json)

Confidence: 40

Reasoning

Disassembly contains PLT stubs and one large user function error@@Base along with its prologue error@@Base-0xa60.

Full CFG reconstruction for the very large error@@Base (thousands of instructions) is complex due to size and indirect control flow; partial reconstruction focuses on identified entry points and sample branches.

Basic blocks derived from branch targets (je, jmp, jne etc.) and function starts; edges are intraprocedural only within identified functions.

Many PLT entries are simple stubs with limited internal CFG.

B.114 Grok – pdpmake (pdpmake_01.json)

Confidence: 45

Reasoning

The disassembly presents a large .text section primarily labeled under error@@Base, indicating either a single dominant function or symbol overlay on multiple routines.

Multiple entry points visible via standard prologues (endbr64, push/pop sequences, stack adjustments) allow identification of distinct functions such as error handlers, init/fini, and utility subs.

Basic blocks derived from branch targets, conditional jumps (je/jne), unconditional jumps, calls (retained in BB), and ret instructions; interprocedural edges omitted per rules.

Truncation in provided disassembly limits full recovery, yielding partial CFG focused on prominent visible control flows in early .text.

Nodes use error@@Base::B¡n¿ prefix for main blob and sub_ prefixes for smaller identifiable routines; edges reflect only intraprocedural transfers.

B.115 Grok – pdpmake (pdpmake_02.json)

Confidence: 45

Reasoning

The disassembly primarily consists of PLT stubs and one large function labeled error@@Base at 0x23b0 containing the bulk of the program's logic.

Multiple small helper-like routines and entry points (e.g., at 0x25c0, 0x2660) are identifiable as separate functions based on prologues and labels.

Basic blocks were identified by locating branch targets, conditional jumps, returns, and fall-through sequences within the .text section; full enumeration is limited by input size but key intraprocedural flows recovered.

No interprocedural edges included; calls to PLT functions do not terminate BBs or create edges.

Partial CFG focused on main function and visible subs for soundness.

B.116 Grok – pdpmake (pdpmake_03.json)

Confidence: 45

Reasoning

The disassembly primarily consists of PLT stubs for library functions and one very large function labeled error@@Base spanning most of the .text section.

Basic blocks were identified by scanning for branch targets (labels like ¡error@@Base+0xNNN¿), conditional jumps (je, jne, etc.), unconditional jumps, calls (which do not terminate BBs), and returns.

PLT entries are treated as trivial functions with single basic blocks. The large error@@Base function has numerous internal basic blocks connected by identified intraprocedural branches; not all edges recovered due to truncation and complexity.

No interprocedural edges or call/return modeling per rules. CFG focuses on identifiable intraprocedural flows within recovered functions.

B.117 Grok – wak (wak_00.json)

Confidence: 45

Reasoning

The disassembly primarily contains one large function labeled error@@Base spanning most of .text, with numerous internal branches, calls, and conditional jumps.

PLT stubs and small init/fini sections have simple linear CFGs with few branches.

Basic blocks identified by sequential instructions terminating at jumps, calls (kept in BB), returns, or branch targets.

Due to truncation and volume, partial reconstruction focuses on major control points in error@@Base and entry points; many edges recoverable from visible je/jne/jmp/ret.

No interprocedural edges; only intra-function CFGs.

B.118 Grok – wak (wak_01.json)

Confidence: 65

Reasoning

Disassembly shows multiple functions in .text section starting at addresses like 0x2390, 0x2470, 0x25d0, 0x26c0 (likely main), and others up to ~0x10f60.

Basic blocks identified by branch targets (e.g., je, jne, jmp), function prologues (push rbp, sub rsp), and returns. PLT stubs treated as trivial functions.

Intraprocedural edges from conditional branches (je/jne at offsets like 0x23b8, 0x24f4), unconditional jmps, and fall-throughs within visible functions. Full CFG partial due to truncation but covers key control flow.

No interprocedural edges added; calls ignored per rules. Node names use generic 'sub_X' or inferred 'main' where clear.

B.119 Grok – wak (wak_02.json)

Confidence: 65

Reasoning

Disassembly contains standard ELF sections with PLT stubs and .text code.

Multiple functions identifiable via prologues (push rbp etc.) and ret instructions at addresses like 0x25d0 (_start-like), 0x26c0 (main), and smaller helpers at 0x2390, 0x2470.

Basic blocks constructed by linear sequences terminated by branches/jumps/returns/calls where control flow changes; intraprocedural only, no call/return edges between functions.

Large binary with truncation limits full enumeration, but key control flows in entry points recovered; nodes use sub_ addresses where names unavailable.

B.120 Grok – wak (wak_03.json)

Confidence: 45

Reasoning

Disassembly contains multiple functions in .text section including error handlers at 0x2390 and 0x2470, _start at 0x25d0, and a large main-like function at 0x26c0 handling getopt and processing logic.

Basic blocks identified by branch targets (e.g., je, jne, jmp) and function entry points. Truncation and size limit full reconstruction; focused on prominent control flows.

Intraprocedural only: no call/return edges between functions. Nodes follow ¡function¿::B¡n¿ format using recovered labels where possible (e.g., start, error_print). Partial but sound CFG from visible branches.

Ethics

Based on the USENIX Security Symposium's policy on ethical considerations [23].

Within up to one page, explain the ethical considerations of your work. You are expected to complete a stakeholder-based ethics analysis or justify and complete an alternative *recognized* approach to considering ethics. For a stakeholder-based ethics analysis, you should identify the relevant stakeholders, consider how your work impacts them (positively or negatively), and discuss any steps you have taken to mitigate negative impacts. If your work involves human subjects, you should also discuss how you have ensured that your work complies with relevant ethical guidelines and regulations (e.g., obtaining informed consent, ensuring privacy and confidentiality, etc.). You may also discuss any broader societal implications of your work. If your work does not involve human subjects or have any ethical implications, please provide a brief justification for this conclusion.

For more details on how to complete a stakeholder-based ethics analysis, see, for instance, the USENIX Security Symposium's guidelines on ethical considerations [23].

C Ethical Considerations Appendix

This research does not involve human participants, personal data, or experiments requiring ethical approval.

The primary stakeholders are reverse engineers and software developers. The proposed evaluation framework may benefit these stakeholders by providing a more systematic methodology for assessing the structural quality of LLM-generated control-flow graphs and by improving understanding of the strengths and limitations of current LLM-based reverse engineering techniques.

The thesis does not introduce new reverse engineering capabilities or methods for bypassing security mechanisms.

To promote transparency and reproducibility, all experiments were conducted using publicly available tools, open-source software projects, and documented methodologies. The evaluation pipeline, generated datasets, and experimental results are made available through the accompanying research repository to facilitate verification and future research.

D AI Usage Considerations Appendix

Artificial intelligence (AI) tools were used during the development of this thesis in accordance with the university's guidelines. AI assistance was used for two primary purposes:

- **Code generation and debugging:** AI-assisted code generation was used to develop, debug, and refine portions of the experimental pipeline and supporting scripts. All generated code was manually reviewed, tested, and validated before being incorporated into the research workflow.
- **Editorial assistance:** AI was used to improve the clarity, grammar, readability, and academic style of the written thesis. All technical content, methodology, experimental design, analysis, interpretations, and conclusions were critically reviewed and verified by the author, who remains fully responsible for the accuracy and integrity of the final manuscript.

No AI-generated text, code, or analysis was included without human review and verification. All cited sources were consulted directly, and responsibility for the content, correctness, and conclusions of this thesis rests solely with the author.

Artifacts

Based on the standard Artifact Appendix template used by the USENIX Security Symposium.

Please follow the self-contained instructions below and assume (i) you are compiling the final version of the Artifact Appendix (no need to cater to reviewers, but to general users of your artifact) and (ii) you are documenting everything in scope for all the artifact evaluation badges (*Artifacts Available*, *Artifacts Functional*, *Results Reproduced*). More information at [24].

E Artifact Appendix

E.1 Abstract

This artifact contains the complete pipeline used to evaluate control-flow graph (CFG) reconstruction using LLVM, Ghidra, rizin, and three Large Language Models (DeepSeek-V3, Gemini 3.5 Flash, and Grok 4).

The repository includes the benchmark programs, dataset generation pipeline, CFG extraction scripts, LLM prompt, evaluation scripts, generated CFGs, and plotting scripts required to reproduce the experimental results presented in this thesis.

E.2 Description & Requirements

This section describes the hardware, software, and benchmark requirements necessary to reproduce the experiments presented in this thesis.

E.2.1 Security, Privacy, and Ethical Concerns. The artifact performs static analysis only and does not execute the benchmark programs during evaluation. No destructive operations, privilege escalation, or security-sensitive actions are performed.

Internet connectivity is required when reproducing the LLM experiments because the evaluated models are accessed through their respective GUIs.

E.2.2 How to Access. The complete artifact is publicly available at:

https://github.com/yavuzk28/Bachelor-Thesis

The repository contains all scripts, benchmark programs, generated data, and documentation necessary to reproduce the evaluation.

E.2.3 Hardware dependencies. The artifact has modest hardware requirements.

- Modern x86-64 processor
- Minimum 8 GB RAM (16 GB recommended)

- Approximately 5 GB free disk space
- No GPU required

E.2.4 Software Dependencies. The following software is required.

- Ubuntu 24.04 LTS (or compatible Linux distribution)
- Python 3.14+
- Java JDK 21+
- Bash (Linux)
- LLVM/Clang
- Ghidra
- rizin
- NetworkX
- NumPy
- Matplotlib
- Git
- objdump
- ctags
- cloc

All required Python packages can be installed using the supplied `requirements.txt` file.

E.2.5 Benchmarks. The evaluation uses 10 open-source C programs:

- lsh
- md5-c
- 2048.c
- fe
- kilo
- Tinyhttpd
- Notepad–
- bzip2
- pdpmake
- wak

Each benchmark is compiled using LLVM/Clang with optimization levels `O0`, `O1`, `O2`, and `O3`.

E.3 Set-up

E.3.1 Installation. Clone the repository and install the required dependencies.

```
git clone https://github.com/yavuzk28/Bachelor-Thesis.git

cd Bachelor-Thesis

python3 -m venv venv

source venv/bin/activate

pip install -r requirements.txt
```

LLVM/Clang, Ghidra, and rizin must be installed according to their official installation instructions.

E.3.2 Basic Test. A basic functionality test can be executed using

```
python3 produce.py
```

Successful execution downloads the benchmark repositories, compiles all benchmark programs, generates LLVM IR, LLVM bitcode (BC), stripped binaries, binary disassembly, sampled disassembly, and extracts CFGs using LLVM, Ghidra, and rizin.

E.4 Evaluation Workflow

E.4.1 Major Claims.

(C1) Traditional binary analysis frameworks reconstruct substantially larger and more structurally detailed control-flow graphs than current Large Language Models.
This claim is supported by the quantitative evaluation presented in Chapter 5.

(C2) Compiler optimization has only a limited influence on the relative behaviour of both traditional CFG extraction frameworks and Large Language Models.
This claim is supported by the optimization-level evaluation and discussed in Section 6.3.

(C3) Although current Large Language Models demonstrate a conceptual understanding of control-flow graph construction, they do not consistently reconstruct CFGs that are structurally comparable to those produced by established binary analysis frameworks.
This claim is supported by the qualitative evaluation presented in Section 6.4.

E.4.2 Experiments.

(E1) CFG Reconstruction Estimated resources: 90 minutes human time, 60 minutes compute time.
Preparation
Execute the dataset generation pipeline:

```
python3 produce.py
```

This automatically downloads the benchmark repositories, compiles all benchmark programs, generates LLVM IR, LLVM bitcode (BC), binary disassembly, sampled disassembly, and extracts CFGs using LLVM, Ghidra, and rizin.
Execution
Manually submit each generated assembly listing to the corresponding LLM using the standardized prompt provided in the repository. Copy the generated CFGs and accompanying reasoning into the provided JSON files. This process is repeated for all benchmark programs, optimization levels, and evaluated LLMs.
Results
The generated CFGs are used for the quantitative evaluation in Chapter 5, while the accompanying reasoning is used for the qualitative analysis in Section 6.4.

(E2) Optimization-Level Evaluation Estimated resources:
2 minutes human time, 2 minutes compute time.
Preparation

Use the generated CFGs from all optimization levels.

Execution

Execute the evaluation pipeline:

```
python3 evaluate.py
```

Results

The evaluation pipeline extracts the LLM reasoning into LaTeX file, computes all graph metrics, and generates the figures presented in the thesis. These outputs reproduce the quantitative evaluation reported in Chapter 5 and the qualitative reasoning summaries discussed in Section 6.4.

E.5 Notes on Reusability

The artifact has been designed to facilitate future experimentation.

Researchers can extend the evaluation by:

- adding additional benchmark programs,
- evaluating alternative Large Language Models,
- integrating different compiler toolchains,
- evaluating additional processor architectures, and
- incorporating new graph metrics or similarity measures.

E.6 Version

Based on the LaTeX template for Artifact Evaluation V20231005. Submission, reviewing and badging methodology followed for the evaluation of this artifact can be found at https://secartifacts.github.io/usenixsec2024/.